

SUPPORT DE COURS COMPLET

Audit organisation et technique

16h CM · 8h TP · 24h

Dr. Zakaria Sawadogo

École Polytechnique de Ouagadougou



Sommaire

Syllabus

1. Chapitre 1 — Introduction à l'audit de sécurité
2. Chapitre 2 — Audit organisationnel : normes et référentiels
3. Chapitre 3 — Audit technique : méthodologie des tests d'intrusion
4. Chapitre 4 — Audit des infrastructures réseau et systèmes
5. Chapitre 5 — Audit des applications et du code
6. Chapitre 6 — Rapport d'audit et plan de remédiation

Audit organisation et technique

Volume horaire : 16h CM · 8h TP (24h)

Objectifs pédagogiques

- Distinguer audit organisationnel et audit technique, et situer chacun dans une démarche globale de gestion de la sécurité.
- Connaître les principaux référentiels et normes utilisés en audit (ISO 27001/27002, PCI-DSS, ANSSI, NIST).
- Mener une démarche d'audit technique (tests d'intrusion) selon une méthodologie structurée (reconnaissance, scan, exploitation, reporting).
- Rédiger un rapport d'audit exploitable par une direction et par des équipes techniques.

Prérequis

Notions de base en réseaux (TCP/IP) et en systèmes d'exploitation ; le cours *Algorithmique et Programmation en C* n'est pas un prérequis strict mais la lecture de scripts est un atout.

Plan de séances

Cours magistraux (16h)

Séance	Chapitre	Durée
1	Introduction à l'audit de sécurité	2h
2	Audit organisationnel : normes et référentiels	3h
3	Audit technique : méthodologie des tests d'intrusion	3h
4	Audit des infrastructures réseau et systèmes	3h
5	Audit des applications et du code	3h
6	Rapport d'audit et plan de remédiation	2h

Travaux pratiques (8h)

Séance	Sujet	Durée
1	Audit organisationnel sur cas d'étude (ISO 27001)	2h
2	Reconnaissance et scan réseau (Nmap)	2h

Séance	Sujet	Durée
3	Test d'intrusion applicatif (OWASP Top 10)	2h
4	Rédaction d'un rapport d'audit complet	2h

Évaluation

- Participation et livrables de TP : 30 %
- Étude de cas notée (analyse d'un référentiel) : 20 %
- Examen final (méthodologie + étude de cas d'audit) : 50 %

Environnement technique des TP

Machine virtuelle Kali Linux ou équivalent, cible d'entraînement légale (Metasploitable2, OWASP Juice Shop, DVWA) déployée en local — **tout test d'intrusion hors de ce cadre pédagogique nécessite une autorisation écrite explicite**, rappelée en séance 1.

Bibliographie

- ISO/IEC 27001:2022, ISO/IEC 27002:2022.
- ANSSI, *Guide d'hygiène informatique*.
- PTES (Penetration Testing Execution Standard) — pentest-standard.org.
- OWASP Testing Guide.

Chapitre 1 — Introduction à l'audit de sécurité

1. Définition

Un audit de sécurité est une évaluation systématique et indépendante du niveau de sécurité d'un système d'information, visant à vérifier sa conformité à un référentiel (normatif, réglementaire ou contractuel) et à identifier les écarts entre l'état constaté et l'état attendu.

Trois mots de cette définition méritent d'être précisés, car ils distinguent un audit d'une simple prestation technique :

- **Systématique** : l'audit suit une méthodologie reproductible (checklist, référentiel, guide de test) et non une exploration au hasard. Deux auditeurs différents, appliquant la même méthodologie sur le même périmètre, doivent aboutir à des constats globalement comparables.
- **Indépendante** : l'auditeur ne doit pas être juge et partie. Un administrateur système qui « audite » sa propre configuration peut avoir des angles morts (biais de confirmation, réticence à pointer ses propres manquements) ; c'est pourquoi les référentiels de certification exigent souvent une séparation entre l'équipe qui exploite le système et l'équipe qui l'audite, voire un auditeur externe.
- **Référentiel** : un audit ne se fait jamais « dans l'absolu ». Il compare toujours un existant à un cadre de référence explicite — une norme (ISO/IEC 27001), un standard technique (CIS Benchmarks), une réglementation (RGPD) ou un cahier des charges contractuel. Sans référentiel affiché, un rapport d'audit n'est qu'une opinion, difficile à contester mais aussi difficile à faire reconnaître par un tiers (assureur, régulateur, client).

Il est utile de distinguer l'audit de sécurité d'activités voisines avec lesquelles il est parfois confondu :

- Le **test d'intrusion (pentest)** est une composante technique de l'audit, centrée sur la démonstration d'exploitabilité, mais un audit peut être bien plus large (revue organisationnelle, revue de configuration sans exploitation active).
- Le **contrôle de conformité (compliance check)** vérifie mécaniquement des cases d'une checklist réglementaire ; un audit de sécurité va au-delà en évaluant l'efficacité réelle des mesures, pas seulement leur existence formelle.
- La **veille de vulnérabilités** (scan récurrent, bulletin de sécurité) est un flux d'information continu, alors que l'audit est un exercice ponctuel et structuré, avec un début, une fin et un livrable.

2. Pourquoi auditer ?

- **Conformité réglementaire** : obligations légales (protection des données, secteurs régulés — banque, santé).
- **Exigence contractuelle** : un client ou partenaire exige un niveau de sécurité prouvé (ex. certification ISO 27001, conformité PCI-DSS pour le paiement en ligne).
- **Gestion des risques** : identifier les vulnérabilités avant qu'un attaquant ne les exploite.
- **Amélioration continue** : mesurer les progrès entre deux audits successifs.

Au-delà de ces motivations « officielles », un auditeur doit garder à l'esprit que la commande d'un audit répond souvent à un déclencheur concret : préparation d'une certification, exigence d'un investisseur ou d'un assureur cyber, suite à un incident de sécurité (même mineur), changement de RSSI, ou encore obligation contractuelle imposée par un grand donneur d'ordre à ses sous-traitants. Comprendre ce déclencheur aide l'auditeur à calibrer le ton du rapport et les priorités de la restitution : un audit demandé après un incident attend des réponses rapides et concrètes, tandis qu'un audit de certification attend une couverture exhaustive du référentiel, y compris sur des points formels.

Il faut aussi rappeler ce qu'un audit **n'est pas** : ce n'est pas une garantie d'absence de vulnérabilité (un audit couvre un périmètre et une période donnés, avec les limites de temps et d'outillage propres à toute prestation), et ce n'est pas un exercice à charge contre les équipes techniques. Un audit mal compris par les équipes auditées génère de la défiance et nuit à la qualité des entretiens et de la collecte d'information ; une bonne communication en amont sur les objectifs de l'audit (amélioration, pas sanction) conditionne largement sa réussite.

3. Les grandes familles d'audit

Type d'audit	Objet	Exemple de livrable
Audit organisationnel	politiques, procédures, gouvernance	rapport de conformité ISO 27001
Audit technique	configuration des systèmes, réseaux, applications	rapport de test d'intrusion
Audit de conformité	respect d'une réglementation précise	rapport PCI-DSS, RGPD
Audit de code	revue du code source	rapport SAST (analyse statique)
Audit physique	sécurité des locaux, contrôle d'accès	rapport d'inspection physique

Ce module couvre principalement les deux premières familles, qui se complètent : un système peut être « techniquement » bien configuré mais dépourvu de procédures (pas de gestion des correctifs formalisée), ou disposer de belles procédures jamais appliquées en pratique.

Ces familles ne sont pas des catégories imperméables ; en pratique, une mission d'audit combine souvent plusieurs volets. Un audit de conformité PCI-DSS, par exemple, embarque à la fois des exigences organisationnelles (politique de sécurité formalisée, gestion des accès) et des exigences techniques précises (segmentation réseau, chiffrement des données de carte, scans de vulnérabilités trimestriels). De même, un audit physique n'est pas anecdotique en sécurité de l'information : un attaquant qui accède physiquement à une salle serveur non protégée, ou qui retrouve des documents sensibles jetés sans destruction (technique dite du *dumpster diving*), contourne d'un coup toutes les mesures logiques mises en place. Un auditeur expérimenté garde toujours à l'esprit cette complémentarité et signale, même hors périmètre strict de sa mission, les risques évidents relevant d'une autre famille d'audit.

Approfondissement : audit vs évaluation continue

La tendance actuelle en cybersécurité est de compléter les audits ponctuels (photographie à un instant t) par une évaluation continue : scans de vulnérabilités automatisés récurrents, plateformes de gestion de la surface d'attaque externe (*Attack Surface Management*), programmes de *bug bounty*. L'audit périodique garde toutefois un rôle irremplaçable : il mobilise un regard humain et expert, capable d'identifier des failles logiques ou organisationnelles qu'aucun outil automatisé ne détecte, et il produit un livrable formel opposable (utile pour une certification, un dossier d'assurance ou une preuve de diligence raisonnable).

4. Les acteurs et postures

- **Auditeur interne** : appartient à l'organisation auditée, connaît le contexte, indépendance relative.
- **Auditeur externe** : société tierce, plus grande indépendance, souvent exigée par les référentiels de certification.
- **Boîte noire (black box)** : l'auditeur ne dispose d'aucune information préalable, simule un attaquant externe.
- **Boîte grise (grey box)** : informations partielles (comptes utilisateurs, documentation), simule une menace interne ou un attaquant ayant déjà un premier accès.
- **Boîte blanche (white box)** : accès complet (code source, architecture, comptes admin), maximise la couverture de l'audit.

Le choix entre ces trois modes d'accès n'est pas qu'une question de réalisme du scénario : c'est aussi un arbitrage coût/couverture. Un audit en boîte noire se rapproche le plus d'une attaque réelle mais consomme une part importante du temps disponible en pure reconnaissance, au détriment de l'analyse en profondeur ; à budget de temps égal, il couvre donc généralement moins de vulnérabilités qu'un audit en boîte blanche. À l'inverse, un audit en boîte blanche maximise la couverture mais peut manquer certains constats liés à l'exposition réelle du système (ce qu'un attaquant externe verrait en premier). Un bon cadrage d'audit précise explicitement le mode retenu et le justifie par rapport à l'objectif de la mission :

Objectif de la mission	Mode d'accès généralement recommandé
Simuler une attaque externe réaliste, tester la détection	boîte noire
Simuler un employé malveillant ou un compte compromis	boîte grise
Certification, revue de sécurité exhaustive avant mise en production	boîte blanche
Audit de code source d'une application critique	boîte blanche (obligatoire pour le SAST et la revue manuelle)

Piège courant : confondre indépendance et compétence

Un auditeur externe n'est pas automatiquement plus compétent qu'une équipe interne — il est simplement structurellement plus indépendant du résultat de l'audit (il n'a pas construit le système qu'il

évalue, et sa rémunération ne dépend pas d'un satisfecit). Il reste indispensable de vérifier les compétences et références de l'auditeur (certifications reconnues comme OSCP, CEH, ou l'agrément PASSI dans le cadre du référentiel ANSSI) : l'indépendance garantit l'absence de conflit d'intérêt, pas la qualité technique du travail rendu.

5. Cadre légal et éthique

Tout audit technique — en particulier un test d'intrusion — implique des actions qui, sans autorisation, seraient qualifiées d'accès et de maintien frauduleux dans un système de traitement automatisé de données. Un audit ne peut être mené que dans le cadre d'un **mandat écrit** précisant :

- le périmètre exact autorisé (adresses IP, applications, plages horaires) ;
- les techniques autorisées et interdites (ex. déni de service exclu par défaut) ;
- les responsables à contacter en cas d'incident pendant l'audit ;
- les modalités de confidentialité et de destruction des données collectées.

Ce document est généralement appelé **règles d'engagement** (*rules of engagement*) et doit être signé par une personne réellement habilitée à autoriser des tests sur le périmètre concerné — pas seulement un interlocuteur technique de bonne volonté. Dans le cas fréquent d'une infrastructure hébergée chez un tiers (fournisseur Cloud, hébergeur mutualisé), l'autorisation du seul client ne suffit généralement pas : la plupart des fournisseurs Cloud imposent une procédure de notification préalable, voire une autorisation explicite de leur part, avant tout test d'intrusion sur les ressources qu'ils hébergent, sous peine de suspension du compte ou de poursuites.

D'autres bonnes pratiques éthiques et contractuelles complètent le mandat :

- **Clause de non-divulgation (NDA)** : l'auditeur a nécessairement accès à des informations sensibles (failles non corrigées, données de configuration, parfois des données à caractère personnel rencontrées incidemment) ; leur confidentialité doit être contractualisée.
- **Divulgation responsable** : si l'audit révèle une vulnérabilité critique susceptible d'affecter des tiers (ex. faille dans un composant largement diffusé), l'auditeur doit suivre un processus de divulgation coordonnée plutôt qu'une publication immédiate, afin de laisser le temps de corriger.
- **Fenêtre de test et point de contact d'urgence** : convenir d'une plage horaire et d'un contact joignable en cas d'incident (service dégradé, alerte déclenchée chez un partenaire de supervision) évite qu'un test légitime ne déclenche une réponse à incident disproportionnée côté client.

Piège courant : le périmètre implicite

Une erreur fréquente, y compris chez des auditeurs expérimentés, consiste à considérer que « tout ce qui appartient à l'entreprise » est dans le périmètre. Or une même adresse IP ou un même nom de domaine peut être partagé avec des tiers (hébergement mutualisé, sous-traitant, filiale non couverte par le mandat) : tester au-delà du périmètre écrit, même de bonne foi, expose l'auditeur à une responsabilité pénale et civile. En cas de doute sur l'appartenance d'un actif au périmètre, la règle est simple : ne pas tester et faire confirmer par écrit auprès du donneur d'ordre.

6. Le cycle de vie d'un audit

1. **Cadrage** : définition du périmètre, des objectifs, du mandat.
2. **Collecte d'information / reconnaissance**.
3. **Analyse** (organisationnelle et/ou technique).
4. **Identification et qualification des écarts/vulnérabilités**.
5. **Rédaction du rapport** (constats, criticité, recommandations).
6. **Restitution** et plan d'action.
7. **Suivi** (contre-audit / vérification de la remédiation).

Ce cycle, bien qu'illustré ici de façon linéaire, est en pratique itératif : l'analyse d'un premier constat peut relancer une phase de collecte d'information complémentaire (par exemple, la découverte d'un serveur non documenté oblige à revenir en reconnaissance), et la rédaction du rapport fait souvent remonter des questions qui nécessitent de revérifier un constat avant publication. Un point souvent sous-estimé par les étudiants est l'étape de **cadrage** : c'est elle qui détermine la durée réaliste de la mission, les ressources nécessaires et les livrables attendus. Un cadrage bâclé (périmètre flou, objectifs mal définis) est la cause la plus fréquente d'audits qui dépassent leur budget de temps ou qui livrent un rapport qui ne répond pas aux attentes du commanditaire.

Chaque phase produit généralement un artefact intermédiaire qui alimente les suivantes : le cadrage produit le mandat et les règles d'engagement ; la reconnaissance produit une cartographie du périmètre (inventaire d'actifs, schéma d'architecture reconstitué) ; l'analyse produit une liste brute de constats ; la qualification y ajoute criticité et preuve ; le rapport en fait une synthèse actionnable. Les chapitres suivants de ce module détaillent chacune de ces phases : le chapitre 2 pour l'audit organisationnel, les chapitres 3 à 5 pour les différentes déclinaisons de l'audit technique, et le chapitre 6 pour la rédaction du rapport et le suivi de la remédiation.

À retenir

- Un audit combine une dimension organisationnelle et une dimension technique, qui se complètent plus qu'elles ne se substituent l'une à l'autre.
- Le mandat écrit et les règles d'engagement sont un prérequis non négociable à toute activité d'audit technique, y compris vis-à-vis d'un fournisseur Cloud tiers.
- Le mode d'accès (black/grey/white box) détermine la couverture et le réalisme de l'audit : il s'agit d'un arbitrage explicite à documenter dans le cadrage, pas d'un simple détail technique.
- L'indépendance de l'auditeur (interne ou externe) garantit l'absence de conflit d'intérêt, mais ne remplace pas la vérification de sa compétence.
- Le cycle de vie de l'audit est itératif : un audit qui ne prévoit pas de phase de suivi (contre-audit) reste une photographie isolée sans garantie de progrès réel.

Chapitre 2 — Audit organisationnel : normes et référentiels

1. ISO/IEC 27001 et 27002

- **ISO/IEC 27001** : norme certifiable définissant les exigences d'un Système de Management de la Sécurité de l'Information (SMSI). Structurée autour du cycle **PDCA** (Plan-Do-Check-Act).
- **ISO/IEC 27002** : catalogue de mesures de sécurité (bonnes pratiques) organisées en thèmes (organisationnel, humain, physique, technologique) que l'ISO 27001 référence.

La distinction entre ces deux normes est une source fréquente de confusion chez les auditeurs débutants : l'ISO 27001 est une norme de **management** — elle décrit un système de pilotage (comment identifier les risques, décider des mesures à appliquer, vérifier leur efficacité, s'améliorer) — alors que l'ISO 27002 est une bibliothèque de **mesures concrètes** (contrôles) dans laquelle on peut piocher pour construire la « Déclaration d'applicabilité » (*Statement of Applicability*, SoA) exigée par l'ISO 27001. Une organisation peut être **certifiée ISO 27001** (le système de management est audité et jugé conforme), mais on ne « certifie » pas l'ISO 27002 en tant que telle : elle sert de référence de bonnes pratiques, y compris hors de toute démarche de certification.

Le cycle **PDCA** structure la logique d'amélioration continue du SMSI :

- **Plan** : établir le contexte, apprécier les risques, définir la politique de sécurité et les objectifs.
- **Do** : mettre en œuvre les mesures de traitement du risque retenues (contrôles techniques et organisationnels).
- **Check** : surveiller et mesurer l'efficacité du SMSI (audits internes, indicateurs, revue de direction).
- **Act** : corriger les non-conformités identifiées et faire évoluer le SMSI en continu.

Clauses principales de l'ISO 27001 (4 à 10)

Clause	Contenu
4	Contexte de l'organisation
5	Leadership (engagement de la direction, politique SSI)
6	Planification (appréciation et traitement des risques)
7	Support (ressources, compétences, sensibilisation)
8	Fonctionnement (mise en œuvre opérationnelle)
9	Évaluation des performances (audit interne, revue de direction)
10	Amélioration (non-conformités, actions correctives)

Un audit organisationnel vérifie, pour chaque clause, l'existence de **preuves documentaires** (politiques, procédures) et de **preuves de mise en œuvre effective** (entretiens, échantillons de tickets,

journaux d'activité).

Ces sept clauses (4 à 10) constituent le corps normatif « obligatoire » de l'ISO 27001 : une organisation candidate à la certification doit démontrer sa conformité à chacune d'elles, sans exception, quelle que soit sa taille ou son secteur. C'est ce qui différencie l'ISO 27001 de nombreux autres référentiels sectoriels dont certaines exigences peuvent être exclues selon le contexte. L'annexe A de la norme (dans sa version 2022, réorganisée en quatre thèmes : organisationnel, humain, physique, technologique) liste les 93 contrôles de référence — c'est cette annexe qui reprend, en version condensée, le contenu détaillé de l'ISO 27002. Un point technique important pour l'auditeur : une organisation peut légitimement exclure un contrôle de l'annexe A de sa Déclaration d'applicabilité, à condition de justifier cette exclusion (par exemple, les contrôles relatifs au développement logiciel sécurisé peuvent être hors périmètre pour une organisation qui n'a aucune activité de développement en interne) — l'auditeur doit vérifier que chaque exclusion est argumentée et cohérente avec le contexte réel de l'organisation, et non utilisée pour esquiver un point sensible.

Piège courant : confondre certification et sécurité réelle

Une organisation certifiée ISO 27001 n'est pas nécessairement « sécurisée » au sens absolu : la certification atteste que le système de management fonctionne (les risques sont identifiés, traités, suivis), pas que le niveau de risque résiduel est nul. Une organisation peut être parfaitement certifiée tout en ayant accepté consciemment un risque élevé sur un actif secondaire, si cette acceptation est documentée et validée par la direction dans le cadre du processus de traitement du risque. L'auditeur doit donc toujours distinguer, dans son évaluation, la conformité au **processus** de management du risque et le niveau de risque **résiduel** effectivement accepté.

2. Autres référentiels usuels

Référentiel	Portée	Usage typique
PCI-DSS	données de cartes bancaires	commerçants et prestataires de paiement
NIST Cybersecurity Framework	gestion du risque cyber (Identify, Protect, Detect, Respond, Recover)	référence internationale, souvent utilisée hors certification
ANSSI (guides et référentiels)	administrations et OIV en France, largement repris en Afrique francophone	hygiène informatique, PASSI (prestataires d'audit)
RGPD	protection des données personnelles	conformité juridique, articulé avec la sécurité technique
CIS Controls	mesures techniques priorisées	audit technique et durcissement (<i>hardening</i>)

Il est utile de comprendre la nature différente de chacun de ces référentiels, car cela conditionne la façon dont un auditeur doit les mobiliser :

- **PCI-DSS** est un standard **contractuel** porté par les grands réseaux de cartes bancaires (et non par un organisme de normalisation international) : il est très prescriptif (règles précises de segmentation réseau, de chiffrement, de rétention de logs) et son non-respect peut entraîner la perte du droit à traiter des paiements par carte, indépendamment de toute action judiciaire. Il distingue plusieurs niveaux de conformité selon le volume de transactions traitées, avec des modalités d'évaluation différentes (auto-évaluation via questionnaire ou audit par un évaluateur qualifié QSA).
- Le **NIST Cybersecurity Framework** (CSF) n'est pas certifiable au sens strict : c'est un cadre de référence, structuré en cinq (puis six dans sa version 2.0, avec l'ajout de la fonction *Govern*) fonctions — Identifier, Protéger, Détecter, Répondre, Récupérer — utilisé pour structurer une démarche de gestion du risque cyber et communiquer sur la maturité d'une organisation, souvent en complément d'une norme certifiable comme l'ISO 27001.
- Les référentiels de l'**ANSSI** (guide d'hygiène informatique, référentiel PASSI pour les prestataires d'audit) sont particulièrement structurants dans l'espace francophone : le guide d'hygiène informatique, en particulier, propose une liste de mesures priorisées et accessibles, souvent utilisée comme grille de premier niveau avant une démarche plus lourde de type ISO 27001.
- Le **RGPD** n'est pas à proprement parler un référentiel de sécurité mais un texte réglementaire relatif à la protection des données personnelles ; il impose cependant des obligations de sécurité explicites (article 32 : mesures techniques et organisationnelles appropriées), ce qui en fait un objet d'audit à part entière, souvent mené conjointement par un auditeur sécurité et un correspondant/délégué à la protection des données (DPO).
- Les **CIS Controls** et les **CIS Benchmarks** associés sont des référentiels très opérationnels, orientés configuration technique, qui seront largement mobilisés dans les chapitres consacrés à l'audit technique (chapitres 3 et 4).

Approfondissement : comment choisir le bon référentiel

Face à une organisation qui n'a jamais formalisé de démarche sécurité, l'auditeur doit souvent recommander un référentiel adapté à sa maturité et à son secteur plutôt que d'imposer d'emblée le plus exigeant. Quelques repères pratiques :

- Une petite structure sans obligation réglementaire particulière peut démarrer efficacement avec un guide d'hygiène informatique (type ANSSI) ou les CIS Controls, plus accessibles qu'une certification complète.
- Une organisation qui traite des paiements par carte n'a pas le choix : PCI-DSS s'impose contractuellement dès lors qu'elle stocke, traite ou transmet des données de carte.
- Une organisation qui vise un marché international ou une clientèle exigeante (banque, assurance, grand groupe) a souvent intérêt à viser l'ISO 27001, référentiel le plus reconnu et le plus universellement audité par les tiers.
- Ces référentiels ne sont pas exclusifs : une même organisation peut être certifiée ISO 27001, se référer au NIST CSF pour communiquer avec des partenaires anglophones, et appliquer PCI-DSS sur son périmètre de paiement.

3. Méthodologie d'un audit organisationnel

1. **Revue documentaire** : politiques de sécurité, chartes, procédures de gestion des incidents, plan de continuité d'activité (PCA/PRA).
2. **Entretiens** avec les parties prenantes (RSSI, DSI, RH, métiers) pour vérifier l'application réelle des procédures.
3. **Échantillonnage** : contrôle de tickets, de comptes utilisateurs, de journaux, de contrats prestataires.
4. **Analyse d'écart (gap analysis)** entre l'état constaté et les exigences du référentiel.
5. **Notation de maturité**, souvent sur une échelle (ex. modèle CMMI adapté : 0 = inexistant à 5 = optimisé).

Chacune de ces étapes appelle des précisions méthodologiques importantes pour un auditeur en formation.

La revue documentaire ne se limite pas à vérifier qu'un document existe : il faut vérifier sa date de dernière mise à jour (une politique de sécurité qui n'a pas été révisée depuis cinq ans est un signal d'alerte, même si son contenu reste globalement pertinent), son niveau d'approbation (a-t-elle été validée par la direction, ou reste-t-elle un brouillon jamais formellement adopté ?) et sa diffusion effective (les collaborateurs concernés savent-ils qu'elle existe et où la trouver ?).

Les entretiens sont l'outil principal pour distinguer le document de la pratique réelle. Un bon auditeur prépare une grille de questions par référentiel et par interlocuteur, mais reste capable de rebondir sur les réponses (« Pouvez-vous me montrer un exemple concret ? », « Que se passe-t-il si un ticket de demande d'accès n'est pas traité dans le délai prévu ? »). Les questions ouvertes révèlent davantage que les questions fermées : demander « comment gérez-vous le départ d'un collaborateur ? » donne beaucoup plus d'information que « avez-vous une procédure de départ ? » (à laquelle la réponse sera presque toujours « oui »).

L'échantillonnage est indispensable car il est rarement possible (ni utile) de vérifier 100 % des cas : sur une population de 500 comptes utilisateurs, un auditeur sélectionnera par exemple un échantillon représentatif (comptes à privilèges, comptes créés récemment, comptes de prestataires externes) plutôt que de tout contrôler. La méthode d'échantillonnage doit être documentée dans le rapport, afin que le lecteur comprenne la portée réelle du constat (« sur un échantillon de 20 comptes à privilèges contrôlés, 6 n'avaient pas fait l'objet d'une revue depuis plus d'un an » est plus informatif et plus honnête que « la revue des comptes est insuffisante »).

L'analyse d'écart consiste à confronter systématiquement chaque exigence du référentiel à l'état constaté ; elle se matérialise généralement dans un tableau de correspondance (souvent appelé *compliance matrix*).

La notation de maturité permet de synthétiser un grand nombre de constats en une vision d'ensemble exploitable par la direction, à condition que l'échelle utilisée soit clairement définie et appliquée de façon cohérente d'un domaine à l'autre.

Une échelle de maturité détaillée (exemple)

Niveau	Libellé	Description
0	Inexistant	Aucune mesure, aucune conscience du risque associé
1	Initial / ad hoc	Mesures ponctuelles, non formalisées, dépendantes d'individus
2	Reproductible	Pratiques répétées mais peu documentées, pas de contrôle systématique
3	Défini	Procédures documentées, diffusées et appliquées de façon cohérente
4	Maîtrisé	Efficacité mesurée par des indicateurs, écarts détectés et corrigés
5	Optimisé	Amélioration continue, benchmarking, anticipation des risques émergents

Piège courant : l'entretien complaisant

Un piège classique de l'audit organisationnel est l'entretien où l'interlocuteur, consciemment ou non, présente une version idéalisée des pratiques. Pour limiter ce biais, l'auditeur doit systématiquement croiser les déclarations des entretiens avec des preuves tangibles (extraits de journaux, captures d'écran d'outils de gestion des accès, tickets réellement clôturés) plutôt que de se satisfaire d'une affirmation verbale. La règle d'or de l'audit — « *trust, but verify* » (faire confiance, mais vérifier) — s'applique particulièrement à ce type d'entretien.

4. Exemple de grille d'évaluation simplifiée

Domaine	Exigence	Constat	Niveau de maturité (0-5)
Gestion des accès	Revue périodique des droits	Aucune revue formalisée depuis 18 mois	1
Gestion des correctifs	Politique de patch management documentée	Politique existe, appliquée à 60 % des serveurs	2
Sensibilisation	Formation annuelle du personnel	Formation dispensée à l'embauche uniquement	2

Une bonne pratique consiste à accompagner chaque ligne de ce type de grille d'une **preuve** (référence au document consulté, extrait d'entretien, capture d'écran) et d'une **recommandation** rattachée directement à l'écart constaté, plutôt que de renvoyer systématiquement vers une synthèse générale en fin de rapport. Voici un exemple enrichi pour la première ligne du tableau ci-dessus :

- **Preuve** : entretien avec le responsable des systèmes le [date], confirmé par l'absence de tout compte rendu de revue des droits dans le registre de gouvernance fourni.
- **Risque associé** : des comptes obsolètes (anciens employés, anciens prestataires) peuvent conserver des accès actifs, augmentant la surface d'attaque et le risque d'accès non autorisé.
- **Recommandation** : instaurer une revue trimestrielle des droits d'accès, formalisée par un compte rendu signé du responsable de chaque périmètre applicatif, avec un focus prioritaire sur

les comptes à privilèges.

5. Risques d'un audit organisationnel mal mené

- Se limiter à la lecture des documents sans vérifier l'application réelle (« conformité de façade »).
- Confondre existence d'une procédure et efficacité de la procédure.
- Absence de priorisation : toutes les non-conformités ne présentent pas le même risque.

À ces trois risques classiques s'ajoutent d'autres écueils fréquents chez les auditeurs en formation :

- **Le formalisme excessif** : produire un rapport de 150 pages qui reprend mécaniquement chaque clause du référentiel sans hiérarchisation ni synthèse exploitable, au détriment de la lisibilité pour la direction.
- **L'oubli du contexte métier** : recommander une mesure de sécurité techniquement irréprochable mais totalement disproportionnée par rapport à la taille, au budget ou à l'activité réelle de l'organisation auditée (par exemple, exiger d'une PME de dix salariés un centre opérationnel de sécurité 24/7).
- **La confusion entre non-conformité et vulnérabilité** : une non-conformité organisationnelle (absence de procédure documentée) n'est pas automatiquement synonyme de vulnérabilité exploitable ; il faut relier explicitement l'écart organisationnel au risque technique concret qu'il engendre pour que le constat soit compréhensible et actionnable.

À retenir

- L'audit organisationnel évalue des processus et de la documentation, pas uniquement des configurations techniques ; il repose sur trois sources de preuve complémentaires — documents, entretiens et échantillons — qu'il faut systématiquement croiser.
- ISO 27001 (norme de management, certifiable) et ISO 27002 (catalogue de mesures) sont complémentaires et se combinent souvent avec des référentiels sectoriels (PCI-DSS) ou nationaux (ANSSI) selon le contexte de l'organisation auditée.
- Le choix du référentiel doit être adapté à la maturité, au secteur et aux obligations contractuelles ou réglementaires de l'organisation ; il n'existe pas de référentiel universellement optimal.
- Une non-conformité doit toujours être qualifiée par un niveau de criticité, une preuve tangible et une recommandation actionnable — jamais énoncée comme une simple observation isolée.
- La certification à un référentiel atteste la conformité d'un processus de management du risque, pas l'absence de risque résiduel : l'auditeur doit distinguer ces deux notions.

Chapitre 3 — Audit technique : méthodologie des tests d'intrusion

1. Le PTES (Penetration Testing Execution Standard)

Le PTES structure un test d'intrusion en sept phases :

1. **Pre-engagement** : cadrage, périmètre, mandat, règles d'engagement.
2. **Intelligence gathering** (reconnaissance) : collecte passive et active d'informations.
3. **Threat modeling** : identification des actifs critiques et des scénarios d'attaque plausibles.
4. **Vulnerability analysis** : identification des failles (scan, analyse manuelle).
5. **Exploitation** : tentative d'exploitation contrôlée des vulnérabilités identifiées.
6. **Post-exploitation** : évaluation de l'impact réel (élévation de privilèges, pivot, exfiltration simulée).
7. **Reporting** : rédaction du rapport.

Le PTES n'est pas la seule méthodologie de référence en test d'intrusion, mais c'est l'une des plus complètes et des plus pédagogiques pour un cours d'introduction, car elle explicite des phases souvent négligées par les praticiens pressés — en particulier le *threat modeling* et le *pre-engagement*. D'autres cadres méthodologiques existent et sont utiles à connaître :

- **OSSTMM** (Open Source Security Testing Methodology Manual) : très orienté mesure quantitative de la sécurité (notion de *RAV*, *Risk Assessment Value*).
- **OWASP Testing Guide** : spécifique aux applications web, largement mobilisé au chapitre 5 de ce module.
- **NIST SP 800-115** : guide technique publié par le NIST, souvent cité en référence dans les cahiers des charges d'audit publics.

Ces méthodologies partagent une structure globale similaire (cadrage, reconnaissance, analyse, exploitation, restitution) : l'essentiel n'est pas de mémoriser un vocabulaire propre à chacune, mais de comprendre la logique commune qui les sous-tend et de savoir l'adapter au contexte de la mission.

Le threat modeling, une phase souvent négligée

La modélisation des menaces (phase 3) consiste à se poser, avant même de lancer le premier scan, la question : « si j'étais un attaquant motivé, que chercherais-je à atteindre dans ce système, et par quel chemin le plus probable ? » Cette réflexion évite à l'auditeur de disperser son temps limité sur des cibles secondaires et l'aide à prioriser : un serveur de test isolé du réseau de production mérite moins d'attention qu'un serveur d'authentification centralisé, même si le second présente, en apparence, moins de vulnérabilités techniques immédiates. Des approches structurées comme **STRIDE** (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege) peuvent être mobilisées pour systématiser cette réflexion, notamment lors d'un audit d'architecture ou de conception.

2. Reconnaissance passive vs active

Type	Exemples de techniques	DéTECTABLE par la cible ?
Passive	OSINT (WHOIS, réseaux sociaux, moteurs de recherche, theHarvester), consultation de code source public	non
Active	scan de ports, résolution DNS directe, requêtes HTTP vers la cible	oui, potentiellement journalisé

La reconnaissance passive mérite d'être développée car elle est souvent sous-estimée par les débutants, pressés d'en venir au scan technique. Une bonne reconnaissance passive peut révéler, sans jamais émettre le moindre paquet vers la cible :

- l'architecture technique probable (offres d'emploi mentionnant des technologies précises, dépôts de code publics laissés ouverts par erreur) ;
- des informations sur les collaborateurs (utile pour évaluer le risque d'ingénierie sociale et pour construire des listes de noms d'utilisateurs plausibles) ;
- des sous-domaines et adresses IP historiques (certificats TLS archivés dans des journaux de transparence, archives de pages web) ;
- des fuites de données antérieures potentiellement réutilisables (mots de passe, adresses e-mail).

```
# Exemples d'outils et de commandes de reconnaissance passive
whois exemple-pedagogique.tld
theHarvester -d exemple-pedagogique.tld -b google,bing
dig exemple-pedagogique.tld ANY
```

La reconnaissance active, à l'inverse, laisse des traces dans les journaux de la cible (pare-feu, IDS/IPS, journaux applicatifs) : c'est un point important à documenter dans le rapport (les événements générés par l'audit peuvent ensuite être comparés aux alertes effectivement déclenchées côté client, ce qui donne une indication indirecte sur les capacités de détection de l'organisation — même si un test d'intrusion n'est pas conçu, en soi, pour évaluer la détection de façon rigoureuse, contrairement à un exercice Red Team).

Piège courant : négliger la reconnaissance pour « aller vite »

Un auditeur pressé qui saute directement au scan de vulnérabilités automatisé prend le risque de manquer des informations qu'aucun scanner ne trouvera seul (identifiants exposés par erreur dans un dépôt public, sous-domaine oublié hébergeant une ancienne version vulnérable de l'application). L'expérience montre qu'une reconnaissance soignée, même chronophage, est souvent l'étape la plus rentable en termes de vulnérabilités critiques découvertes par heure investie.

3. Scan et énumération

- **Scan de ports** (`nmap`) : identifier les services exposés (ports ouverts, fermés, filtrés).

- **Fingerprinting** : identifier la version des services et systèmes (`nmap -sV -O`).
- **Énumération** : lister les ressources exposées (partages réseau, utilisateurs, répertoires web).

```
nmap -sV -sC -p- 192.168.1.10
```

Cette commande mérite d'être décomposée, car chaque option a un rôle précis dans une démarche d'audit :

- `-sV` : détection de version des services (utile pour ensuite confronter chaque version à une base de CVE) ;
- `-sC` : exécution des scripts par défaut de la bibliothèque NSE (*Nmap Scripting Engine*), qui réalisent des vérifications complémentaires (bannières, certificats TLS, configurations faibles connues) ;
- `-p-` : balaie l'intégralité des 65 535 ports TCP, plutôt que la liste restreinte des ports les plus courants scannée par défaut — indispensable en audit, car un service critique mal exposé sur un port non standard est un constat fréquent et significatif.

D'autres options sont fréquemment utiles selon le contexte de la mission :

```
# Scan UDP (souvent négligé, plus lent, mais révèle des services comme SNMP, DNS, NTP)
nmap -sU --top-ports 100 192.168.1.10

# Scan discret (moins susceptible de déclencher une alerte, mais plus lent)
nmap -sS -T2 192.168.1.10

# Scan de vulnérabilités via scripts NSE dédiés
nmap --script vuln 192.168.1.10
```

L'énumération, une fois les services identifiés, cherche à en extraire un maximum d'informations exploitables : liste des partages réseau accessibles (SMB), utilisateurs valides sur un service d'authentification, répertoires et fichiers accessibles sur un serveur web (`gobuster`, `ffuf`, `dirb`). Une bonne pratique méthodologique consiste à documenter systématiquement, service par service, ce qui a été identifié — même en l'absence de vulnérabilité immédiate — car cette cartographie servira de base à l'ensemble des phases suivantes et devra figurer, sous une forme synthétique, dans les annexes du rapport final.

4. Analyse de vulnérabilités

Outils automatisés (scanners) confrontent les versions de services identifiées à des bases de vulnérabilités connues (CVE) :

- **Nessus, OpenVAS** : scanners généralistes réseau/systèmes.
- **Nikto, OWASP ZAP, Burp Suite** : scanners orientés applications web.

Un scan automatisé produit des **faux positifs** (vulnérabilité signalée mais non exploitable en pratique) et peut manquer des **faux négatifs** (vulnérabilités logiques non détectables par signature) : il ne remplace jamais l'analyse manuelle d'un auditeur.

Comprendre l'origine de ces faux positifs et faux négatifs aide à calibrer la confiance à accorder à un rapport de scan brut :

- Un **faux positif** survient typiquement lorsque le scanner identifie une version de logiciel signalée comme vulnérable dans sa base de signatures, sans tenir compte d'un correctif appliqué manuellement (rétroportage de correctif, *backport*) qui ne modifie pas le numéro de version affiché — pratique courante sur certaines distributions Linux d'entreprise.
- Un **faux négatif** peut résulter d'une vulnérabilité logique propre à l'application (par exemple, un contrôle d'accès mal conçu permettant à un utilisateur d'accéder aux données d'un autre en modifiant un identifiant dans l'URL) : ce type de faille ne correspond à aucune signature connue et ne peut être détecté que par une analyse manuelle ou une revue de code.

Il est donc essentiel de **trier manuellement** les résultats bruts d'un scanner avant de les inclure dans un rapport : présenter à un client une liste de 200 alertes automatisées non vérifiées, dont une part importante de faux positifs, nuit à la crédibilité de l'auditeur et dilue les constats réellement critiques.

Exemple d'utilisation d'OpenSSL pour vérifier une configuration TLS

Au-delà des scanners dédiés, des outils standards permettent de vérifier manuellement des points de configuration précis, ce qui est utile pour confirmer ou infirmer une alerte automatisée :

```
# Vérifier les protocoles et suites de chiffrement supportés par un serveur
openssl s_client -connect exemple-pedagogique.tld:443 -tls1_2

# Afficher les informations du certificat présenté (validité, émetteur, algorithme)
echo | openssl s_client -connect exemple-pedagogique.tld:443 2>/dev/null | openssl x509
```

Un constat classique d'audit technique consiste, par exemple, à identifier la persistance de protocoles obsolètes comme SSLv3 ou TLS 1.0 encore acceptés par un serveur, ou un certificat expiré ou auto-signé sur un service exposé publiquement — deux non-conformités simples à démontrer avec ce type de commande, et faciles à faire comprendre dans le rapport, y compris à un lecteur non technique.

5. Exploitation contrôlée

L'exploitation vise à démontrer l'impact réel d'une vulnérabilité (preuve de concept), sans dégrader le service ni les données de production. Principes :

- privilégier les preuves non destructives (ex. lecture d'un fichier témoin plutôt que suppression) ;
- documenter précisément chaque action entreprise (horodatage, commande, résultat) pour la traçabilité et le rapport ;
- s'arrêter au périmètre autorisé par le mandat, même si une extension semble techniquement possible.

L'exploitation est souvent la phase la plus valorisée par les apprenants, mais elle doit rester subordonnée aux règles d'engagement définies en amont. Quelques précisions pratiques :

- **Preuve minimale suffisante** : pour une injection SQL, par exemple, il suffit généralement de démontrer l'extraction d'une donnée non sensible (numéro de version de la base de données, nom d'une table) plutôt que d'extraire l'intégralité d'une table contenant des données à caractère personnel — le principe de minimisation s'applique aussi à l'auditeur.
- **Environnement de test dédié** : lorsque le mandat le permet, privilégier un environnement de pré-production ou de test plutôt que la production pour les tentatives d'exploitation les plus risquées (élévation de privilèges, déni de service partiel).
- **Journal d'exploitation (log book)** : tenir, tout au long de la mission, un journal chronologique précis de chaque action technique (commande exécutée, horodatage, résultat, capture d'écran) : ce journal est à la fois une preuve pour le rapport final et une protection pour l'auditeur en cas de contestation ultérieure sur le déroulement de la mission.
- **Communication en cas d'incident imprévu** : si une action, même prudente, provoque un effet inattendu sur la production (service dégradé, alerte de sécurité déclenchée chez le client), l'auditeur doit immédiatement contacter le point de contact prévu au mandat plutôt que de tenter de dissimuler ou corriger seul l'incident.

Piège courant : la surenchère d'exploitation

Un piège fréquent chez les auditeurs juniors est de chercher à « aller le plus loin possible » pour impressionner le commanditaire, au risque de dépasser le périmètre autorisé ou de causer un incident évitable. La valeur d'un audit ne se mesure pas au nombre de systèmes compromis, mais à la pertinence, la clarté et l'actionnabilité des constats produits. Une preuve de concept élégante et non destructive vaut toujours mieux qu'une compromission complète non maîtrisée.

6. Notation de la criticité (CVSS)

Le score **CVSS** (Common Vulnerability Scoring System) permet de qualifier objectivement la gravité d'une vulnérabilité à partir de métriques (vecteur d'attaque, complexité, privilèges requis, impact sur confidentialité/intégrité/disponibilité). Un score CVSS de 9.8 sur 10 (ex. exécution de code à distance sans authentification) impose une remédiation immédiate ; un score de 3.1 peut être traité dans un cycle de correctifs standard.

Le CVSS (dans ses versions 3.1 et 4.0, la version 4.0 affinant notamment la prise en compte du contexte d'exploitation) repose principalement sur un jeu de métriques dites de base (*Base metrics*), calculées à partir de caractéristiques intrinsèques de la vulnérabilité :

Métrique	Ce qu'elle mesure
Vecteur d'attaque (AV)	Depuis où l'attaque est-elle possible : réseau, adjacent, local, physique ?
Complexité d'attaque (AC)	Le succès de l'attaque dépend-il de conditions difficiles à réunir ?
Privilèges requis (PR)	Faut-il déjà disposer d'un compte, et avec quel niveau de privilège ?
Interaction utilisateur (UI)	Une action de la victime est-elle nécessaire (ouvrir un lien, un fichier) ?
Impact sur la confidentialité (C)	Fuite de données possible et étendue

Métrique	Ce qu'elle mesure
Impact sur l'intégrité (I)	Modification non autorisée de données ou de systèmes
Impact sur la disponibilité (A)	Déni de service possible et étendue

Ces métriques de base ne racontent toutefois pas toute l'histoire : le CVSS prévoit également des métriques temporelles (existence d'un exploit public, disponibilité d'un correctif) et environnementales (contexte spécifique du système audité), trop rarement renseignées en pratique alors qu'elles sont précisément ce qui permet d'adapter le score générique à la réalité de l'organisation auditée. C'est un point que le chapitre 6 développera : un score CVSS brut ne doit jamais être utilisé seul pour prioriser une remédiation sans contextualisation métier.

7. Limites et complémentarité avec l'audit organisationnel

Un test d'intrusion technique donne une photographie à un instant t d'un périmètre limité. Il ne dit rien sur la capacité de l'organisation à détecter et répondre à une attaque réelle (à ne pas confondre avec un exercice de type *Red Team*, plus large et plus long, qui teste aussi la détection).

Cette limite mérite d'être explicitée davantage, car elle conditionne le choix du bon type de mission selon l'objectif recherché :

- Un **test d'intrusion classique** vise l'exhaustivité de la découverte de vulnérabilités sur un périmètre donné, dans un temps contraint, généralement avec la connaissance (au moins partielle) des équipes de défense.
- Un exercice **Red Team** vise à tester, sur une durée plus longue, la capacité réelle de détection et de réaction des équipes de défense (*Blue Team*), sans que celles-ci soient nécessairement informées à l'avance ; l'objectif n'est plus de trouver un maximum de vulnérabilités mais d'atteindre un objectif précis (exfiltration simulée d'une donnée cible, par exemple) par le chemin le plus discret possible.
- Un exercice **Purple Team** organise une collaboration active entre attaquants et défenseurs pendant le test, dans un objectif pédagogique et d'amélioration continue plutôt que d'évaluation à l'aveugle.

Par ailleurs, un test d'intrusion technique ne remplace pas un audit organisationnel : une organisation peut réussir un test d'intrusion (aucune vulnérabilité critique exploitable trouvée sur le périmètre testé) tout en présentant des lacunes organisationnelles majeures (absence de plan de réponse à incident, absence de sensibilisation du personnel) qui la rendraient vulnérable à un vecteur non couvert par le test, en particulier l'ingénierie sociale. C'est pourquoi les deux chapitres précédents et les suivants de ce module doivent être lus comme complémentaires plutôt que substituables.

À retenir

- Le PTES structure la démarche en sept phases, de la reconnaissance au rapport ; d'autres méthodologies (OSSTMM, OWASP Testing Guide, NIST SP 800-115) partagent une logique globale similaire.

- La reconnaissance, souvent négligée, est fréquemment l'étape la plus rentable en vulnérabilités critiques découvertes par heure investie ; elle doit toujours précéder le scan automatisé.
- L'automatisation (scanners) accélère l'analyse mais ne remplace pas le jugement de l'auditeur : chaque alerte automatisée doit être vérifiée manuellement avant d'être incluse dans un rapport.
- L'exploitation doit rester strictement subordonnée au mandat et privilégier des preuves non destructives, documentées avec précision (journal d'exploitation).
- Le score CVSS objective la priorisation des vulnérabilités identifiées, mais ses métriques temporelles et environnementales, souvent négligées, sont indispensables pour l'adapter au contexte réel de l'organisation.
- Un test d'intrusion classique, un exercice Red Team et un audit organisationnel répondent à des objectifs différents et complémentaires, à ne jamais confondre.

Chapitre 4 — Audit des infrastructures réseau et systèmes

1. Audit de l'architecture réseau

- **Segmentation** : vérifier l'existence de zones réseau distinctes (DMZ, réseau interne, réseau d'administration) et que le filtrage entre zones suit le principe du moindre privilège.
- **Flux autorisés** : revue des règles de pare-feu (une règle ANY-ANY en sortie ou entrée est presque toujours une non-conformité à documenter).
- **Points d'entrée exposés** : recensement de tout ce qui est accessible depuis Internet (VPN, portails, services mal exposés par erreur).

La segmentation réseau est souvent présentée comme une simple bonne pratique, mais elle joue en réalité un rôle de **limitation de la propagation** (*containment*) qui conditionne fortement l'impact réel d'une compromission. Un attaquant qui parvient à compromettre un poste utilisateur dans un réseau plat (sans segmentation) peut potentiellement atteindre directement les serveurs critiques ; dans une architecture correctement segmentée, ce même attaquant doit franchir plusieurs zones de filtrage successives, ce qui augmente le temps et la difficulté de l'attaque, et donc les chances de détection.

Un principe important à vérifier lors d'un audit de segmentation est la **zone d'administration dédiée** : les interfaces d'administration des équipements (pare-feu, serveurs, hyperviseurs) ne devraient jamais être accessibles depuis le réseau utilisateur standard, mais uniquement depuis un réseau d'administration restreint, idéalement avec un accès filtré par une passerelle d'administration (*bastion* ou *jump host*) journalisant chaque connexion.

Revue des règles de pare-feu : une méthode pas à pas

Une revue de règles de pare-feu suit généralement cette logique :

1. Extraire la configuration complète (export de règles) plutôt que de se fier à une description orale de l'architecture.
2. Identifier les règles trop permissives (ANY en source, destination ou port), en particulier celles ouvertes en sortie vers Internet, souvent négligées alors qu'elles facilitent l'exfiltration de données en cas de compromission.
3. Identifier les règles **obsolètes** : flux autorisés pour un projet ou un prestataire qui n'existe plus, souvent jamais nettoyés faute de processus de revue périodique.
4. Vérifier la cohérence entre la documentation d'architecture attendue et les règles réellement appliquées (un écart entre les deux est un indicateur de dérive de configuration, ou *configuration drift*).
5. Tester, lorsque le mandat le permet, l'efficacité réelle du filtrage depuis différentes zones (un scan de ports lancé depuis le réseau utilisateur vers le réseau d'administration doit ne rien retourner s'il est correctement filtré).

```
# Vérification empirique du filtrage entre deux zones réseau
nmap -Pn -p 22,3389,443,161 10.10.5.0/24
```


Piège courant : la confiance excessive dans le pare-feu périmétrique

Une organisation qui a investi dans un pare-feu périmétrique performant peut avoir tendance à négliger la sécurité interne, en considérant que « tout ce qui est à l'intérieur est de confiance ». Cette hypothèse, héritée d'une vision « château fort » de la sécurité, est de moins en moins tenable : la majorité des incidents graves impliquent un point d'entrée initial limité (poste compromis par hameçonnage, identifiants volés) suivi d'un mouvement latéral à l'intérieur du réseau. C'est le fondement du modèle de sécurité **Zero Trust**, qui recommande de ne faire confiance à aucun flux par défaut, y compris interne, et de vérifier systématiquement l'identité et le contexte de chaque connexion — un point qu'un auditeur peut utilement évoquer dans ses recommandations, même s'il dépasse le strict périmètre technique de l'audit.

2. Audit des équipements réseau

- Mots de passe par défaut non changés (routeurs, switchs, imprimantes réseau, caméras IP).
- Protocoles non chiffrés encore actifs (Telnet, HTTP d'administration, SNMP v1/v2 avec communauté `public`).
- Firmware obsolète et absence de politique de mise à jour.

Ces trois catégories de constats reviennent avec une régularité frappante dans les audits d'infrastructure, ce qui en fait une checklist minimale à ne jamais omettre :

- **Mots de passe par défaut** : de nombreux équipements réseau et IoT (caméras, imprimantes, objets connectés de supervision) sont livrés avec des identifiants par défaut documentés publiquement par le fabricant. Un audit sérieux inclut systématiquement une tentative de connexion avec les identifiants par défaut connus du modèle identifié (fingerprinting préalable), dans le respect du mandat.
- **Protocoles non chiffrés** : Telnet et les interfaces HTTP d'administration transmettent les identifiants en clair, interceptables par tout attaquant en position d'écoute sur le même segment réseau (attaque de type *ARP spoofing* suivie d'une capture de trafic). SNMP en version 1 ou 2c repose sur une « communauté » (chaîne d'authentification) souvent laissée à la valeur par défaut `public` en lecture et parfois `private` en écriture — cette dernière permettant potentiellement de modifier la configuration de l'équipement.
- **Firmware obsolète** : contrairement aux systèmes d'exploitation serveurs, les équipements réseau et IoT font souvent l'objet d'une politique de mise à jour beaucoup moins rigoureuse, alors qu'ils sont directement exposés au trafic réseau. L'auditeur doit vérifier l'existence d'un inventaire des équipements avec leur version de firmware et la fréquence de vérification des mises à jour de sécurité disponibles.

```
# Identification de version SNMP et test de la communauté par défaut (avec autorisation)
snmpwalk -v2c -c public 192.168.1.1 system
```

Bonne pratique : l'inventaire des actifs comme prérequis

Un audit d'infrastructure ne peut être exhaustif que si l'organisation dispose d'un **inventaire à jour** de ses équipements réseau. En son absence, l'auditeur doit reconstruire cet inventaire par la découverte

active (scan réseau), ce qui prend du temps et ne garantit jamais une couverture à 100 % (un équipement éteint pendant l'audit, ou volontairement dissimulé du fait d'un usage non autorisé — phénomène de « shadow IT » — peut échapper à la découverte). Recommander la mise en place et la maintenance d'un inventaire d'actifs figure généralement, à juste titre, parmi les toutes premières recommandations d'un rapport d'audit d'infrastructure, car cet inventaire conditionne l'efficacité de tous les contrôles ultérieurs (gestion des correctifs, détection d'intrusion, réponse à incident).

3. Audit des systèmes d'exploitation (hardening)

Points de contrôle typiques, alignés sur les référentiels de durcissement (CIS Benchmarks) :

Domaine	Points de contrôle
Comptes et authentification	comptes par défaut désactivés, politique de mots de passe, MFA pour les comptes à privilèges
Services	services inutiles désactivés (réduction de la surface d'attaque)
Correctifs	politique de patch management, délai moyen d'application des correctifs critiques
Journalisation	logs activés, centralisés, conservés une durée suffisante
Droits	principe du moindre privilège, séparation des comptes admin/utilisateur

Les **CIS Benchmarks** (publiés par le Center for Internet Security) sont des guides de configuration très détaillés, déclinés par système d'exploitation et par version (Windows Server, distributions Linux courantes, équipements réseau, bases de données, conteneurs). Chaque point de contrôle y est présenté avec une justification technique, une procédure de vérification et une procédure de remédiation, ce qui en fait un outil particulièrement utile pour un auditeur : plutôt que de réinventer une checklist, il est possible de s'appuyer directement sur ces benchmarks et, pour les systèmes les plus courants, sur des outils d'évaluation automatisée qui en vérifient l'application (scanners de conformité de configuration).

Quelques points d'attention à approfondir par domaine :

- **Comptes et authentification** : au-delà de la simple existence d'une politique de mots de passe, l'auditeur doit vérifier sa robustesse réelle (longueur minimale, complexité, historique empêchant la réutilisation) et surtout l'application effective du MFA (authentification multifacteur) sur les comptes à privilèges — un point de contrôle devenu quasi incontournable dans les référentiels récents, tant les identifiants volés ou devinés restent un vecteur d'attaque initial fréquent.
- **Services** : chaque service actif est une surface d'attaque potentielle. La désactivation des services inutiles (principe de réduction de la surface d'attaque, ou *attack surface reduction*) est une mesure simple, peu coûteuse et à fort impact, souvent négligée par manque de temps lors du déploiement initial des systèmes.
- **Correctifs** : au-delà de l'existence d'une politique, l'indicateur le plus révélateur est le **délai moyen d'application** des correctifs critiques (*Mean Time To Patch*) ; un audit peut utilement échantillonner un ensemble de serveurs et comparer leur niveau de correctifs à la date de publication des correctifs de sécurité correspondants.

- **Journalisation** : des journaux existent souvent localement, mais leur valeur pour la détection et l'investigation dépend de leur **centralisation** (SIEM ou solution équivalente) et de leur durée de conservation, qui doit être compatible avec le délai moyen de détection d'une intrusion (souvent de plusieurs semaines à plusieurs mois lorsque l'attaquant reste discret) et avec les obligations réglementaires applicables.
- **Droits** : le principe du moindre privilège s'applique autant aux comptes humains qu'aux comptes de service utilisés par les applications ; un compte de service disposant de droits d'administrateur du domaine pour exécuter une tâche planifiée simple est un constat classique et à fort impact.

Exemple de commande d'audit local (Linux)

```
# Lister les comptes disposant d'un shell interactif (candidats à vérifier)
awk -F: '$7 ~ /(bash|sh)$/ {print $1}' /etc/passwd

# Vérifier les services en écoute sur le système
ss -tulnp

# Vérifier la configuration SSH (points fréquemment non conformes)
grep -Ei 'PermitRootLogin|PasswordAuthentication|Protocol' /etc/ssh/sshd_config
```

Ces trois commandes illustrent une démarche simple d'audit local : identifier les comptes exploitables interactivement, cartographier les services réellement exposés, et vérifier des paramètres de configuration critiques (connexion root directe en SSH, authentification par mot de passe plutôt que par clé) qui figurent systématiquement dans les CIS Benchmarks correspondants.

4. Cas particulier : audit d'un domaine Active Directory

L'annuaire Active Directory est une cible privilégiée car il centralise l'authentification. Points d'audit fréquents :

- comptes avec mot de passe n'expirant jamais ou marqués `PASSWD_NOTREQD` ;
- délégation Kerberos non contrainte mal maîtrisée (risque d'usurpation) ;
- groupes à privilèges (`Domain Admins`) surdimensionnés ;
- absence de séparation entre comptes d'administration et comptes utilisateurs quotidiens ;
- outils dédiés d'audit : `BloodHound` (cartographie des chemins d'attaque), `PingCastle` (score de maturité AD).

Active Directory mérite ce traitement particulier car il concentre, dans la quasi-totalité des environnements d'entreprise sous Windows, un rôle d'infrastructure critique disproportionné par rapport à l'attention qu'il reçoit souvent lors des audits généralistes. Une poignée de mauvaises configurations, prises isolément anodines, peuvent se combiner en un **chemin d'attaque** menant directement à la compromission de l'ensemble du domaine (*Domain Admin*) — c'est précisément ce que des outils comme `BloodHound` permettent de visualiser, en modélisant l'annuaire sous forme de graphe de relations (appartenance à des groupes, droits délégués, sessions actives) et en identifiant automatiquement les chemins les plus courts vers les comptes à privilèges les plus élevés.

Quelques constats classiques à approfondir :

- **Kerberoasting** : tout compte de service disposant d'un *Service Principal Name* (SPN) est susceptible de voir son ticket Kerberos extrait par un utilisateur authentifié standard, puis soumis à une attaque hors ligne par force brute si son mot de passe est faible — d'où l'importance de mots de passe longs et aléatoires pour les comptes de service, idéalement gérés par un coffre-fort de secrets.
- **Délégation Kerberos non contrainte** : un compte configuré avec ce type de délégation peut, dans certaines conditions, capturer et réutiliser les tickets d'authentification d'autres utilisateurs se connectant à lui, ce qui en fait une cible de choix si un attaquant en obtient le contrôle.
- **Groupes à privilèges surdimensionnés** : le groupe `Domain Admins` ne devrait contenir qu'un nombre restreint de comptes dédiés à l'administration, utilisés ponctuellement, et jamais de comptes utilisateurs du quotidien (consultation de messagerie, navigation web) — la pratique consistant à donner un compte unique « tout puissant » à chaque administrateur reste malheureusement fréquente et constitue un facteur aggravant majeur en cas de compromission d'un seul poste de travail.

Bonne pratique : le modèle en tiers (tiering)

Microsoft recommande un modèle d'administration en **tiers** (*tier model*), qui sépare strictement les comptes et postes d'administration selon le niveau de criticité des ressources administrées (contrôleurs de domaine en tier 0, serveurs en tier 1, postes utilisateurs en tier 2), avec l'interdiction pour un compte d'un tier donné de s'authentifier sur une ressource d'un tier plus sensible. Vérifier l'existence, même partielle, d'une telle séparation est un point de contrôle avancé mais particulièrement révélateur de la maturité d'une organisation en matière de gestion des identités et des accès.

5. Audit du Cloud et des environnements virtualisés

- Vérification des configurations de stockage (buckets S3 ou équivalents en accès public non justifié).
- Gestion des identités et accès (IAM) : clés d'API non tournées, permissions trop larges.
- Isolation entre locataires (tenants) dans un environnement mutualisé.
- Journalisation et alerte (CloudTrail ou équivalent) activées et supervisées.

L'audit d'environnements Cloud introduit une spécificité importante : le **modèle de responsabilité partagée**. Selon le niveau de service considéré (IaaS, PaaS, SaaS), la répartition des responsabilités de sécurité entre le fournisseur Cloud et le client change fortement. Sur une infrastructure IaaS, le fournisseur sécurise la couche physique et la virtualisation, mais la configuration du système d'exploitation, du réseau virtuel et des applications reste de la responsabilité du client ; sur un service SaaS, le fournisseur gère l'essentiel de la pile technique, et la responsabilité du client se concentre sur la configuration des accès et des paramètres applicatifs. Un audit Cloud doit toujours commencer par clarifier ce partage de responsabilité, faute de quoi l'auditeur risque soit d'auditer des éléments hors du contrôle du client, soit d'en oublier qui sont bien de sa responsabilité.

Quelques approfondissements sur les points de contrôle listés ci-dessus :

- **Stockage en accès public non justifié** : de nombreuses fuites de données documentées publiquement dans l'industrie proviennent d'espaces de stockage objet (type S3) configurés par erreur en accès public, souvent lors d'un déploiement rapide ou d'un test jamais reconfiguré ensuite. La vérification de ce point ne nécessite généralement aucune exploitation active : une simple tentative de lecture non authentifiée suffit à démontrer le constat.
- **Gestion des identités et accès (IAM)** : le Cloud facilite la création rapide de clés d'API et de rôles, mais aussi leur prolifération incontrôlée. Un audit IAM vérifie notamment l'absence de clés d'accès à privilèges larges (* en action et en ressource) attribuées à des services qui n'en ont pas besoin, la rotation régulière des clés statiques, et la préférence donnée aux rôles temporaires plutôt qu'aux identifiants permanents.
- **Isolation entre locataires** : dans un environnement mutualisé, un défaut d'isolation peut permettre à un client d'accéder, même involontairement, aux ressources d'un autre client du même fournisseur — un point difficile à auditer directement sans la coopération du fournisseur, mais qui doit au minimum faire l'objet d'une vérification contractuelle et documentaire (certifications du fournisseur, rapports d'audit tiers type SOC 2).
- **Journalisation et alerte** : contrairement à une infrastructure classique où les journaux doivent être configurés service par service, les fournisseurs Cloud proposent généralement un service de journalisation centralisé des actions d'administration (traçant qui a fait quoi, quand, depuis où) ; un constat fréquent est que ce service, bien que disponible, n'est simplement pas activé, ou activé sans alerte associée en cas d'action sensible (suppression d'un journal, modification d'une politique de sécurité).

6. Restitution d'un audit d'infrastructure

Chaque constat technique doit être formulé selon la structure : **observation** (fait constaté et preuve), **risque** (ce que cela permet à un attaquant), **recommandation** (action corrective priorisée), **référence** (CIS Benchmark, ISO 27002, CVE le cas échéant).

Cette structure en quatre temps mérite d'être illustrée par un exemple complet, transposable à n'importe quel constat de ce chapitre :

Observation : le service SNMP du commutateur cœur de réseau (adresse 192.168.1.1) répond en version 2c avec la communauté par défaut `public` en lecture, comme démontré par la commande `snmpwalk` exécutée le [date]. **Risque** : un attaquant en position d'accès au réseau interne peut, sans authentification robuste, extraire des informations de configuration détaillées de l'équipement (table de routage, interfaces, parfois mots de passe faiblement protégés selon le modèle), facilitant la cartographie de l'infrastructure en vue d'une attaque ultérieure. **Recommandation** : désactiver SNMP v1/v2c au profit de SNMP v3 (authentification et chiffrement), ou à défaut remplacer la communauté par défaut par une valeur forte et restreindre l'accès SNMP par liste de contrôle d'accès (ACL) au seul poste de supervision légitime. **Référence** : CIS Benchmark pour équipements réseau ; ISO/IEC 27002, thème technologique — sécurité des réseaux.

Cette structure systématique facilite non seulement la lecture du rapport, mais aussi son exploitation ultérieure par les équipes techniques chargées de la remédiation, qui peuvent directement transformer chaque constat en tâche opérationnelle (voir chapitre 6).

À retenir

- L'audit d'infrastructure combine architecture réseau, configuration système et, de plus en plus, environnements Cloud/AD ; la segmentation réseau reste un mécanisme fondamental de limitation de la propagation en cas de compromission.
- Les référentiels de durcissement (CIS Benchmarks) fournissent des points de contrôle réutilisables et mesurables, applicables aussi bien aux systèmes d'exploitation qu'aux équipements réseau, bases de données ou conteneurs.
- Active Directory concentre un rôle critique disproportionné : un audit dédié, appuyé sur des outils comme BloodHound ou PingCastle, permet d'identifier des chemins d'attaque invisibles au niveau de constats isolés.
- L'audit Cloud impose de clarifier au préalable le modèle de responsabilité partagée, faute de quoi l'auditeur risque d'évaluer des éléments hors du contrôle réel du client ou d'en oublier qui restent de sa responsabilité.
- Chaque constat doit relier un fait technique à un risque métier compréhensible par la direction, en suivant systématiquement la structure observation / risque / recommandation / référence.

Chapitre 5 — Audit des applications et du code

1. Pourquoi auditer les applications spécifiquement

Les applications web et métier constituent la surface d'attaque la plus exposée et la plus évolutive (déploiements fréquents, code sur mesure). Ce chapitre prépare le cours *Sécurité des applications*, qui approfondit chaque type de vulnérabilité.

Cette spécificité mérite d'être expliquée : contrairement à un équipement réseau ou à un système d'exploitation, dont la sécurité repose largement sur des configurations standardisées et des correctifs fournis par l'éditeur, une application métier développée sur mesure ne bénéficie d'aucun filet de sécurité équivalent. Chaque ligne de code métier est une occasion potentielle d'introduire une vulnérabilité, et cette surface évolue à chaque nouvelle fonctionnalité livrée — dans un contexte d'intégration et de déploiement continu (CI/CD), il n'est pas rare qu'une application en production évolue plusieurs fois par semaine, voire par jour. Cette caractéristique explique pourquoi l'audit applicatif ne peut se limiter à un contrôle ponctuel annuel : il doit s'articuler avec des contrôles automatisés intégrés au cycle de développement, un point développé plus loin dans ce chapitre (section 3).

Un autre facteur d'exposition est la nature même des applications web : elles sont, par construction, accessibles à un grand nombre d'utilisateurs, parfois anonymes, ce qui en fait une cible de choix pour des attaques automatisées à grande échelle (recherche opportuniste de vulnérabilités connues sur des composants tiers largement diffusés) autant que pour des attaques ciblées.

2. OWASP Top 10 comme grille d'audit

Le classement OWASP Top 10 recense les catégories de vulnérabilités applicatives les plus critiques et sert de checklist minimale pour tout audit applicatif :

1. Contrôle d'accès défaillant (*broken access control*)
2. Défaillances cryptographiques
3. Injection (SQL, commande, etc.)
4. Conception non sécurisée (*insecure design*)
5. Mauvaise configuration de sécurité
6. Composants vulnérables et obsolètes
7. Défaillances d'identification et d'authentification
8. Défaillances d'intégrité des données et logiciels
9. Journalisation et surveillance insuffisantes
10. Falsification de requête côté serveur (SSRF)

L'OWASP (*Open Web Application Security Project*) publie et met à jour périodiquement ce classement à partir de données agrégées auprès d'organisations et de contributeurs de la communauté ; il ne s'agit pas d'un palmarès figé mais d'un instantané évolutif des catégories de risque jugées les plus significatives à un moment donné. Pour un auditeur, l'intérêt principal du Top 10 n'est pas tant la liste

elle-même — qui reste une catégorisation de haut niveau — que la richesse documentaire qui l'accompagne (guide de test associé, exemples de code vulnérable et corrigé, références vers des ressources complémentaires), largement mobilisée dans la conception de méthodologies de test détaillées.

Il est utile de préciser certaines catégories qui prêtent parfois à confusion :

- **Contrôle d'accès défaillant** (catégorie n°1 dans les versions récentes du classement) regroupe des défauts très divers : accès direct à une ressource par manipulation d'identifiant dans l'URL (IDOR, *Insecure Direct Object Reference*), élévation de privilèges horizontale (accéder aux données d'un autre utilisateur de même niveau) ou verticale (accéder à des fonctionnalités réservées à un rôle supérieur), absence de vérification côté serveur d'une restriction appliquée uniquement côté client.
- **Conception non sécurisée** (*insecure design*) se distingue d'une simple erreur d'implémentation : il s'agit d'un défaut présent dès la phase de conception (par exemple, un mécanisme de récupération de mot de passe reposant sur des questions secrètes devinables), qui ne peut être corrigé par un simple correctif de code mais nécessite de repenser la fonctionnalité elle-même.
- **SSRF** (*Server-Side Request Forgery*) est une catégorie plus récemment mise en avant : elle consiste à forcer le serveur applicatif à émettre, pour le compte de l'attaquant, des requêtes vers des ressources normalement inaccessibles depuis l'extérieur (services internes, métadonnées d'instance Cloud) — une vulnérabilité de plus en plus critique avec la généralisation des architectures Cloud, où les métadonnées d'instance exposent parfois des identifiants temporaires à privilèges élevés.

Piège courant : réduire l'audit applicatif au seul Top 10

Le Top 10 est une base de départ, pas une checklist exhaustive : il ne couvre volontairement que les dix catégories jugées les plus significatives à l'échelle globale, et ne remplace pas une analyse spécifique à la logique métier de l'application audité. Une application de gestion financière, par exemple, peut présenter des vulnérabilités logiques très spécifiques (contournement d'un plafond de transaction, manipulation du calcul d'une remise) qui n'entrent dans aucune des dix catégories au sens strict mais représentent un risque métier tout aussi réel. Un auditeur expérimenté utilise le Top 10 comme point de départ méthodologique, puis l'enrichit systématiquement d'une analyse dédiée à la logique métier propre à chaque application.

3. Approches d'audit applicatif

Approche	Description	Outils typiques
DAST (Dynamic Application Security Testing)	tests en boîte noire sur l'application en fonctionnement	OWASP ZAP, Burp Suite
SAST (Static Application Security Testing)	analyse du code source sans exécution	SonarQube, Semgrep, Bandit (Python), Cppcheck (C/C++)
SCA (Software Composition Analysis)	analyse des dépendances tierces et de leurs vulnérabilités connues	OWASP Dependency-Check, npm audit

Approche	Description	Outils typiques
Revue manuelle de code	lecture experte, notamment pour la logique métier	—

Une démarche mature combine ces approches : le SAST/SCA s'intègre au pipeline CI/CD pour un contrôle continu, tandis que le DAST et la revue manuelle interviennent à intervalles réguliers ou avant une mise en production majeure.

Chacune de ces quatre approches présente des forces et des limites qu'un auditeur doit connaître pour les combiner efficacement plutôt que de considérer qu'une seule suffit :

- Le **DAST** teste l'application « de l'extérieur », comme le ferait un attaquant réel, sans connaissance du code source. Son avantage est de refléter fidèlement ce qui est réellement exposé et exploitable ; sa limite est qu'il ne couvre que les chemins d'exécution effectivement atteints pendant le test (un formulaire non découvert par le crawler de l'outil ne sera jamais testé) et qu'il peine à détecter des vulnérabilités sans effet observable immédiat.
- Le **SAST** analyse le code source sans l'exécuter, ce qui lui permet de couvrir l'intégralité du code, y compris les chemins rarement empruntés en pratique ; en contrepartie, il génère généralement davantage de faux positifs qu'un DAST bien calibré, car il manque le contexte d'exécution réel (une donnée signalée comme non filtrée par l'analyse statique peut, en pratique, être filtrée par un composant tiers non analysé).
- Le **SCA** répond à un problème spécifique et de plus en plus critique : la majorité du code d'une application moderne provient de bibliothèques tierces (open source ou commerciales), dont les vulnérabilités connues (CVE) sont documentées dans des bases publiques. Un SCA compare l'inventaire des dépendances utilisées (souvent formalisé dans une nomenclature logicielle, ou *Software Bill of Materials*, SBOM) à ces bases pour signaler les composants à mettre à jour.
- La **revue manuelle de code** reste irremplaçable pour la logique métier et les vulnérabilités de conception, qu'aucun outil automatisé ne peut détecter par nature, faute de comprendre l'intention fonctionnelle du code.

```
# Exemple d'utilisation d'un outil SCA en ligne de commande
npm audit --audit-level=high

# Exemple d'analyse SAST légère avec Semgrep sur un dépôt de code
semgrep --config=auto ./src
```

Bonne pratique : intégrer les contrôles dans le pipeline CI/CD (DevSecOps)

L'intégration du SAST et du SCA directement dans le pipeline d'intégration continue — de sorte qu'une vulnérabilité critique nouvellement introduite bloque automatiquement la fusion d'une modification de code ou le déploiement — est aujourd'hui considérée comme une pratique de référence, souvent désignée sous le terme de **DevSecOps** (intégration de la sécurité comme responsabilité partagée tout au long du cycle de développement, plutôt que comme une étape isolée en fin de projet). Un audit applicatif peut utilement évaluer, en plus des vulnérabilités elles-mêmes, la maturité de cette intégration : la présence de contrôles automatisés dans le pipeline est un indicateur fort de la capacité de l'organisation à maintenir un niveau de sécurité dans la durée, bien au-delà de l'instant de l'audit.

4. Méthodologie d'un pentest applicatif

1. **Cartographie** de l'application (pages, fonctionnalités, rôles utilisateurs, points d'entrée de données).
2. **Tests d'authentification et de gestion de session** (politique de mot de passe, expiration de session, protection contre le brute force).
3. **Tests de contrôle d'accès** (un utilisateur standard peut-il accéder à des ressources d'un autre utilisateur ou d'un administrateur en modifiant un identifiant dans l'URL — IDOR ?).
4. **Tests d'injection** (SQL, commande système, LDAP) sur chaque point de saisie.
5. **Tests côté client** (XSS stocké/réfléchi, CSRF).
6. **Tests de configuration** (en-têtes de sécurité HTTP, gestion des erreurs, informations sensibles exposées).

Cette méthodologie appelle plusieurs précisions pratiques utiles à un auditeur en formation :

La cartographie (étape 1) doit être réalisée avec au minimum un compte pour chaque rôle applicatif distinct (utilisateur standard, gestionnaire, administrateur), afin de pouvoir ensuite comparer systématiquement ce que chaque rôle peut et ne peut pas atteindre — une étape indispensable pour la détection des défaillances de contrôle d'accès à l'étape 3.

Les tests d'authentification (étape 2) portent notamment sur : la robustesse de la politique de mot de passe (longueur, complexité, vérification contre des listes de mots de passe compromis connus) ; la protection contre les attaques par force brute (verrouillage temporaire de compte, limitation du débit de requêtes) ; la sécurité du mécanisme de récupération de mot de passe (souvent le maillon faible, car conçu comme une voie de contournement de l'authentification normale) ; et la gestion du cycle de vie de la session (expiration après inactivité, invalidation effective du jeton lors de la déconnexion, régénération de l'identifiant de session après authentification pour prévenir la fixation de session).

Les tests de contrôle d'accès (étape 3), souvent réalisés en confrontant systématiquement les requêtes générées par un compte à privilèges élevés à un compte de moindre privilège (remplacement de l'identifiant de session, sans autre modification), sont probablement les plus révélateurs en pratique : les défaillances de contrôle d'accès figurent en tête du classement OWASP Top 10 précisément parce qu'elles sont fréquentes, souvent faciles à exploiter (aucune compétence technique avancée n'est requise, une simple modification d'URL ou de paramètre peut suffire) et à fort impact.

Les tests d'injection (étape 4) doivent être systématiquement appliqués à **chaque point de saisie** de l'application, y compris ceux qui semblent a priori peu exposés (en-têtes HTTP personnalisés, paramètres de tri ou de filtre, champs de recherche, en-têtes `User-Agent` ou `Referer` parfois journalisés puis affichés sans échappement). Un exemple d'extrait de charge utile pédagogique illustrant une tentative de détection d'injection SQL par observation du comportement (méthode dite « à l'aveugle », ou *blind SQL injection*) :

```
# Payload de détection (comportement conditionnel attendu si vulnérable)
' AND 1=1 --
' AND 1=2 --
```

Si l'application renvoie une réponse différente entre ces deux charges utiles (par exemple, un résultat affiché dans le premier cas et une page vide dans le second), cela constitue un indice fort d'injection SQL exploitable, à confirmer ensuite par une preuve de concept plus poussée dans le respect du mandat.

Les tests côté client (étape 5) portent sur les vulnérabilités qui s'exécutent dans le navigateur de la victime plutôt que sur le serveur : le XSS (*Cross-Site Scripting*) stocké est généralement considéré comme plus critique que le XSS réfléchi, car il affecte tous les visiteurs consultant la page compromise, sans nécessiter d'interaction spécifique de la victime (contrairement au XSS réfléchi, qui nécessite généralement que la victime clique sur un lien piégé). Le CSRF (*Cross-Site Request Forgery*) exploite la confiance qu'une application accorde à une session authentifiée, en forçant le navigateur de la victime à émettre, à son insu, une requête vers l'application ciblée.

Les tests de configuration (étape 6) incluent la vérification des en-têtes de sécurité HTTP :

```
# Vérifier les en-têtes de sécurité renvoyés par une application web
curl -I https://exemple-pedagogique.tld
```

Des en-têtes comme `Content-Security-Policy`, `Strict-Transport-Security`, `X-Content-Type-Options` ou `X-Frame-Options` réduisent, sans les éliminer, certaines classes d'attaques côté client ; leur absence n'est généralement pas critique isolément mais constitue un facteur aggravant en présence d'autres vulnérabilités, et leur présence est un indicateur de la maturité générale de l'équipe de développement en matière de sécurité.

Piège courant : tester uniquement les fonctionnalités visibles

Un piège classique consiste à limiter les tests aux fonctionnalités accessibles depuis l'interface utilisateur standard, en négligeant les points d'entrée moins visibles mais tout aussi exploitables : API exposées mais non documentées publiquement (souvent découvertes par l'inspection du trafic réseau généré par l'application), fonctionnalités d'import/export de données, interfaces d'administration accessibles sur un chemin distinct mais non protégées par un contrôle d'accès réseau. Une cartographie rigoureuse (étape 1), incluant l'analyse du trafic réseau généré par l'application dans un navigateur, réduit fortement ce risque d'angle mort.

5. Audit de code : exemple de grille (langage C, en lien avec le module Algorithmique)

Point de contrôle	Risque associé
Utilisation de <code>strcpy</code> , <code>gets</code> , <code>sprintf</code> sans borne	débordement de tampon
Absence de vérification du retour de <code>malloc</code>	déréférencement de pointeur nul
<code>free</code> sans remise à <code>NULL</code>	use-after-free
Chaîne de format non constante passée à <code>printf</code>	vulnérabilité format string

Point de contrôle	Risque associé
Calculs de taille non vérifiés (multiplication avant <code>malloc</code>)	débordement d'entier

Ces cinq points de contrôle, bien que spécifiques au langage C, illustrent une catégorie de vulnérabilités — les erreurs de gestion mémoire — historiquement responsable d'une part très importante des vulnérabilités critiques découvertes dans les logiciels bas niveau (systèmes d'exploitation, navigateurs, serveurs). Ils méritent d'être développés :

- **Débordement de tampon** (*buffer overflow*) : survient lorsqu'une écriture en mémoire dépasse la taille allouée à la structure de destination, pouvant écraser des données adjacentes (variables voisines, adresse de retour d'une fonction sur la pile), ce qui peut, dans le pire des cas, permettre l'exécution de code arbitraire. Les fonctions historiques comme `strcpy`, `gets` ou `sprintf` ne vérifient pas la taille de la destination et doivent être systématiquement remplacées par leurs équivalents bornés (`strncpy`, `fgets`, `snprintf`), eux-mêmes à utiliser avec attention (une troncature silencieuse mal gérée peut créer d'autres classes de bugs).
- **Déréférencement de pointeur nul** : `malloc` retourne `NULL` en cas d'échec d'allocation ; ignorer cette possibilité et déréférencer directement le pointeur retourné provoque un plantage (dénier de service) et, dans certains contextes, peut être combiné à d'autres défauts pour un impact plus grave.
- **Use-after-free** : utiliser une zone mémoire après l'avoir libérée par `free` est un défaut particulièrement dangereux et difficile à détecter par relecture simple du code, car le programme peut continuer à fonctionner apparemment normalement pendant longtemps avant de manifester un comportement erratique ou exploitable ; remettre systématiquement le pointeur à `NULL` après un `free` (et vérifier cette valeur avant toute réutilisation) est une mesure de précaution simple et largement recommandée.
- **Vulnérabilité format string** : passer une donnée contrôlée par l'utilisateur directement comme chaîne de format à `printf` (au lieu de la passer comme argument d'un format constant, par exemple `printf("%s", donnee)` plutôt que `printf(donnee)`) permet potentiellement à un attaquant de lire, voire d'écrire, des zones arbitraires de la mémoire du processus.
- **Débordement d'entier** : un calcul de taille effectué avant un appel à `malloc` (par exemple, `n * taille_element`) peut déborder silencieusement si `n` est contrôlé par un attaquant et suffisamment grand, aboutissant à une allocation plus petite que prévu et donc à un débordement de tampon ultérieur lors du remplissage de la structure.

```
/* Exemple pédagogique : correction d'un point de contrôle classique */
/* Version vulnérable */
char destination[16];
strcpy(destination, source); /* pas de vérification de taille */

/* Version corrigée */
char destination[16];
strncpy(destination, source, sizeof(destination) - 1);
destination[sizeof(destination) - 1] = '\0'; /* garantir la terminaison */
```

Approfondissement : outils d'analyse pour le code bas niveau

Au-delà de la revue manuelle, plusieurs outils facilitent la détection de ces classes de vulnérabilités dans le code C/C++ :

- **Analyseurs statiques** : `Cppcheck`, les avertissements du compilateur activés au niveau maximal (`-Wall -Wextra` avec GCC/Clang), ou des outils plus avancés d'analyse de flux de données.
- **Outils dynamiques d'instrumentation** : `Valgrind` (détection de fuites mémoire et d'accès invalides à l'exécution), `AddressSanitizer` (instrumentation à la compilation détectant débordements et *use-after-free* avec un net gain de performance par rapport à Valgrind).
- **Fuzzing** : technique consistant à soumettre au programme un grand nombre d'entrées aléatoires ou mutées automatiquement, à la recherche de plantages révélateurs de vulnérabilités ; particulièrement efficace sur du code d'analyse de formats de fichiers ou de protocoles réseau, où la surface d'entrées possibles est vaste.

6. Limites

Un audit applicatif ponctuel ne couvre que l'état du code à un instant donné : toute évolution ultérieure du code peut réintroduire des vulnérabilités déjà corrigées, d'où l'intérêt d'intégrer des contrôles automatisés (SAST/SCA) en continu plutôt que de se reposer uniquement sur un audit annuel.

D'autres limites méritent d'être mentionnées pour donner une vision réaliste des attentes qu'un commanditaire peut avoir vis-à-vis d'un audit applicatif :

- **Couverture fonctionnelle incomplète** : dans le temps contraint d'une mission, il est rarement possible de tester exhaustivement toutes les combinaisons de rôles, de fonctionnalités et de données possibles ; l'auditeur doit donc prioriser les fonctionnalités les plus sensibles (paiement, gestion des accès, données personnelles) plutôt que de chercher une couverture uniforme.
- **Dépendance à l'environnement de test fourni** : un audit réalisé sur un environnement de test peut ne pas refléter parfaitement la configuration de production (données de volume différent, intégrations tierces désactivées), ce qui peut masquer certains constats ou, à l'inverse, en révéler qui n'existent pas en production.
- **Vulnérabilités liées à l'écosystème plutôt qu'au code lui-même** : une application peut être parfaitement codée mais reste dépendante de la sécurité de son infrastructure d'hébergement, de ses intégrations tierces et de la configuration de son environnement d'exécution — d'où l'intérêt de croiser les constats de ce chapitre avec ceux du chapitre 4 (audit des infrastructures) pour obtenir une vision complète du risque applicatif.

À retenir

- L'OWASP Top 10 est la référence minimale pour structurer un audit applicatif, mais doit être complété par une analyse spécifique à la logique métier propre à chaque application.
- SAST, DAST et SCA sont complémentaires, pas substituables l'un à l'autre : chacun couvre des angles morts différents, et leur intégration continue au pipeline CI/CD (approche DevSecOps) prolonge l'effet d'un audit ponctuel dans la durée.
- Les défaillances de contrôle d'accès figurent parmi les vulnérabilités les plus fréquentes et les plus faciles à exploiter : elles méritent une attention systématique, y compris sur les fonctionnalités et API les moins visibles.

- L'audit de code bas niveau (C/C++) cible spécifiquement les erreurs de gestion mémoire et de validation des entrées, dont l'impact peut aller du simple plantage à l'exécution de code arbitraire ; des outils dynamiques (Valgrind, AddressSanitizer) et le fuzzing complètent utilement la revue manuelle.
- Un audit applicatif ponctuel reste, par construction, une photographie datée : sa valeur dans la durée dépend de son articulation avec des contrôles automatisés continus.

Chapitre 6 — Rapport d'audit et plan de remédiation

1. Objectifs du rapport

Le rapport est le principal livrable d'un audit : c'est lui qui sera lu, discuté et utilisé pour arbitrer des décisions et des budgets. Un audit techniquement excellent mais mal restitué perd une grande partie de sa valeur.

Cette affirmation, qui peut surprendre des étudiants davantage attirés par la dimension technique de l'audit, mérite d'être développée : dans la grande majorité des organisations, les personnes qui décident d'allouer un budget de remédiation, de recruter un poste supplémentaire en sécurité ou de revoir une architecture ne liront jamais le détail des commandes exécutées ni les preuves de concept techniques. Elles liront la synthèse, regarderont éventuellement un tableau de priorités, et prendront leur décision sur cette base. Un auditeur qui néglige la qualité rédactionnelle de son rapport — même après un travail technique irréprochable — prend le risque que ses constats les plus critiques restent sans suite faute d'avoir été compris et priorisés par les bonnes personnes.

Le rapport remplit en réalité plusieurs fonctions simultanées, qu'il faut garder à l'esprit lors de sa rédaction :

- une fonction **décisionnelle** (aider la direction à arbitrer des priorités et des budgets) ;
- une fonction **opérationnelle** (permettre aux équipes techniques de reproduire, comprendre et corriger chaque constat) ;
- une fonction **de preuve** (démontrer, pour un audit de certification ou une obligation contractuelle, qu'une évaluation a bien été réalisée selon une méthodologie donnée) ;
- parfois une fonction **juridique** (élément de preuve de diligence raisonnable en cas de contentieux ultérieur, notamment après un incident de sécurité).

Ces fonctions multiples expliquent pourquoi un rapport d'audit sérieux se structure toujours en plusieurs niveaux de lecture, du plus synthétique au plus détaillé, plutôt qu'en un document uniforme.

2. Structure type d'un rapport d'audit

1. **Synthèse à destination de la direction (executive summary)** : 1 à 2 pages, sans jargon technique, avec le niveau de risque global et les priorités.
2. **Contexte et périmètre** : rappel du mandat, dates, méthode, limites de l'audit.
3. **Méthodologie** : référentiel utilisé, mode d'accès (black/grey/white box), outils.
4. **Constats détaillés**, un par vulnérabilité ou non-conformité, avec pour chacun :
 5. titre et catégorie (ex. « Injection SQL sur le formulaire de connexion ») ;
 6. criticité (score CVSS ou échelle qualitative Critique/Élevé/Moyen/Faible) ;
 7. description technique et preuve de concept (capture, extrait de log, commande) ;
 8. impact métier ;
 9. recommandation de remédiation, si possible priorisée et estimée en effort.
10. **Synthèse des recommandations** sous forme de tableau priorisé.

11. **Annexes** : détails techniques, sorties d'outils, méthodologie complète.

Chacune de ces sections répond à un besoin de lecture différent, ce qui justifie qu'elles soient nettement séparées plutôt que mélangées.

La synthèse à destination de la direction (section 1) est probablement la partie la plus difficile à rédiger, précisément parce qu'elle doit être courte et dépourvue de jargon technique tout en restant fidèle à la réalité des constats. Une bonne synthèse répond en une à deux pages à trois questions simples : où en est l'organisation en matière de sécurité par rapport au référentiel ou à l'objectif visé ? Quels sont les deux ou trois risques les plus importants à traiter en priorité ? Que faut-il faire, à quel horizon, pour les traiter ? Elle évite absolument le jargon technique non expliqué (un terme comme « SSRF » ou « Kerberoasting » n'a aucune place dans cette section sans reformulation en langage courant) et privilégie des formulations orientées impact métier plutôt que mécanisme technique.

Le contexte et le périmètre (section 2) doivent explicitement mentionner les **limites** de l'audit : ce qui n'a pas été testé (par exclusion contractuelle, par manque de temps, ou parce que hors périmètre), les conditions dans lesquelles les tests ont été réalisés (environnement de test ou de production, disponibilité limitée d'un compte de test) et toute contrainte ayant pu affecter la couverture réelle de l'audit. Cette transparence protège à la fois le commanditaire (qui sait précisément ce qui a été couvert) et l'auditeur (qui ne peut être tenu responsable de vulnérabilités hors du périmètre convenu).

La méthodologie (section 3) doit être suffisamment précise pour qu'un tiers puisse comprendre comment les constats ont été obtenus, sans nécessairement entrer dans le détail exhaustif de chaque commande (réservé aux annexes).

Les constats détaillés (section 4) constituent le cœur technique du rapport. Chaque constat doit être **autoportant** : un lecteur qui ne consulterait que ce constat isolément (par exemple, extrait et transmis à un prestataire externe pour correction) doit pouvoir comprendre le problème, sa gravité et la marche à suivre sans avoir à lire l'intégralité du rapport.

La synthèse des recommandations (section 5) reprend, sous forme de tableau compact, l'ensemble des constats avec leur priorité, ce qui permet un pilotage rapide sans revenir au détail de chaque constat — c'est souvent cette section qui sert directement de base au plan de remédiation traité en section 4 de ce chapitre.

Les annexes (section 6) accueillent tout ce qui alourdirait la lecture du corps du rapport sans en changer la compréhension : sorties brutes d'outils (scans complets), captures d'écran supplémentaires, glossaire technique.

Exemple de constat détaillé rédigé intégralement

Pour illustrer concrètement la structure attendue d'un constat (section 4), voici un exemple pédagogique complet, fictif, construit sur un scénario classique :

C-03 — Absence d'authentification multifacteur sur l'accès VPN administrateur **Catégorie** : Défaillances d'identification et d'authentification (réf. OWASP A07) **Criticité** : Élevée (CVSS 3.1 base score 8.1 — AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N) **Description** : l'accès VPN utilisé par les administrateurs pour l'administration à distance de l'infrastructure ne requiert qu'un identifiant et un

mot de passe, sans second facteur d'authentification. Un test réalisé le [date] avec un compte de test fourni a permis de confirmer l'absence de toute sollicitation de second facteur lors de la connexion. **Impact métier** : en cas de compromission d'un mot de passe administrateur (hameçonnage, réutilisation d'un mot de passe divulgué lors d'une fuite tierce), un attaquant externe pourrait obtenir un accès direct au réseau d'administration, avec un potentiel d'impact majeur sur la confidentialité et l'intégrité de l'ensemble du système d'information. **Recommandation** : déployer une authentification multifacteur (application d'authentification ou clé matérielle) obligatoire pour tous les comptes disposant d'un accès VPN à privilèges administratifs, en priorité absolue avant tout autre chantier de remédiation identifié dans ce rapport. **Effort estimé** : faible à moyen (dépend de la solution MFA déjà disponible dans l'environnement d'identité existant).

3. Qualifier la criticité

Une bonne pratique consiste à combiner **vraisemblance** (facilité d'exploitation, exposition) et **impact** (confidentialité, intégrité, disponibilité, conformité réglementaire) dans une matrice de risque, plutôt que d'utiliser un score technique brut (CVSS) sans contextualisation métier — une vulnérabilité CVSS 9.8 sur un serveur de test isolé n'a pas le même risque réel qu'une vulnérabilité CVSS 6.0 sur le système de paiement en production.

Cette combinaison peut être formalisée dans une **matrice de risque** à deux dimensions, outil visuel simple et largement compris, y compris par un public non technique :

Impact \ Vraisemblance	Faible	Moyenne	Élevée
Faible	Faible	Faible	Moyen
Moyen	Faible	Moyen	Élevé
Élevé	Moyen	Élevé	Critique

L'utilisation d'une telle matrice suppose que l'auditeur documente explicitement, pour chaque constat, les facteurs qui ont motivé le positionnement retenu sur chaque axe :

- pour la **vraisemblance** : la vulnérabilité est-elle exploitable sans authentification préalable ? Nécessite-t-elle des compétences techniques avancées ou un outil public largement disponible ? Le système concerné est-il directement exposé sur Internet ou uniquement accessible depuis un réseau interne déjà restreint ?
- pour l'**impact** : quelles données ou fonctionnalités sont concernées (données personnelles, données financières, fonctionnalité non critique) ? Le système touché est-il en production ou dans un environnement de moindre sensibilité ? Existe-t-il une obligation réglementaire spécifique renforçant l'impact (notification obligatoire en cas de fuite de données personnelles, par exemple) ?

Piège courant : la priorisation uniquement par score CVSS brut

Prioriser un plan de remédiation en triant mécaniquement les constats par score CVSS décroissant, sans tenir compte du contexte métier, est une erreur fréquente chez les auditeurs juniors et chez

certaines outils automatisés qui ne proposent que cette vue par défaut. Le score CVSS de base ne prend pas en compte l'exposition réelle (un service accessible uniquement depuis un réseau d'administration fortement restreint n'a pas le même niveau de risque réel qu'un service identique exposé directement sur Internet), ni la sensibilité des données concernées, ni les mesures compensatoires éventuellement déjà en place (segmentation, surveillance renforcée). Un rapport de qualité explique toujours, pour chaque constat, comment le score technique a été ajusté — à la hausse ou à la baisse — par le contexte métier.

4. Plan de remédiation

Le plan de remédiation transforme chaque recommandation en action suivie :

Constat	Priorité	Action	Responsable	Échéance	Statut
Mot de passe admin par défaut	Critique	Changer et appliquer une politique de mots de passe forts	Équipe infra	J+7	À faire
Absence de MFA sur le VPN	Élevé	Déployer une authentification à double facteur	RSSI	J+30	En cours
Composant obsolète (CVE connue)	Élevé	Mettre à jour la bibliothèque concernée	Équipe dev	J+15	À faire

Un plan de remédiation efficace repose sur quelques principes de gestion de projet appliqués au contexte de la sécurité :

- **Un seul responsable identifié par action** : une action assignée à une « équipe » plutôt qu'à une personne nommée a statistiquement moins de chances d'être menée à terme dans les délais, faute d'appropriation individuelle claire.
- **Des échéances réalistes et différenciées selon la criticité** : une vulnérabilité critique directement exploitable depuis Internet appelle une correction en urgence (heures à quelques jours), tandis qu'une non-conformité organisationnelle de moindre impact peut légitimement s'inscrire dans un cycle de plusieurs mois — le plan doit refléter cette différenciation plutôt que d'imposer un rythme uniforme.
- **Des mesures compensatoires en cas de correction différée** : lorsqu'une correction définitive ne peut être appliquée immédiatement (dépendance à un fournisseur, contrainte de compatibilité), le plan doit prévoir des mesures temporaires réduisant le risque en attendant (restriction d'accès réseau, surveillance renforcée, désactivation temporaire d'une fonctionnalité non essentielle).
- **Un suivi régulier et visible** : un plan de remédiation qui n'est revu qu'à l'occasion de l'audit suivant perd une grande partie de son utilité ; un point de suivi périodique (hebdomadaire pour les actions critiques, mensuel pour les autres) permet de détecter rapidement les retards et d'en comprendre les causes (manque de ressources, dépendance technique bloquante, changement de priorité).

Bonne pratique : documenter les risques acceptés

Il arrive qu'une organisation décide, en toute connaissance de cause, de ne pas corriger un constat donné (coût disproportionné par rapport au risque résiduel, incompatibilité avec une contrainte métier, système en fin de vie déjà planifiée). Ce choix est légitime dans le cadre d'une démarche de gestion du risque mature, à condition d'être **formalisé** : décision documentée, validée par une personne habilitée (généralement le RSSI ou la direction), avec une date de réexamen. Un auditeur qui constate, lors d'un contre-audit, qu'un constat critique n'a pas été corrigé sans qu'aucune décision d'acceptation du risque n'ait été formalisée doit le signaler comme une non-conformité en tant que telle, distincte du constat technique initial.

5. Contre-audit et suivi

Un audit de suivi (souvent partiel, ciblé sur les constats précédents) permet de vérifier l'application effective des corrections. C'est une pratique attendue dans les cycles de certification (audits de surveillance ISO 27001) et une preuve de maturité pour l'organisation audité.

Un contre-audit efficace ne se limite pas à revérifier mécaniquement chaque constat initial : il doit également vérifier que la correction appliquée n'a pas introduit de nouveau problème (une mise à jour de composant, par exemple, peut corriger une vulnérabilité tout en modifiant un comportement fonctionnel de façon non anticipée) et que la correction est **durable**, pas seulement appliquée ponctuellement pour l'occasion du contre-audit (un piège classique consiste, pour l'organisation audité, à corriger temporairement un constat juste avant le contre-audit sans pérenniser la mesure — un contrôle un peu plus tard dans le temps permet de détecter ce type de régression).

Le rythme des contre-audits dépend du contexte : dans un cycle de certification ISO 27001, des audits de surveillance sont généralement réalisés annuellement entre deux audits de certification complets (eux-mêmes réalisés à intervalle pluriannuel selon le schéma de certification retenu) ; en dehors de tout cadre de certification, une organisation mature planifie généralement un contre-audit ciblé sur les constats critiques dans les mois suivant leur identification, puis un audit complet à intervalle régulier (annuel étant une fréquence courante, à adapter selon le niveau de risque et le rythme d'évolution du système d'information).

6. Communication et postures

- Adapter le niveau de détail au public (direction vs équipes techniques) — d'où l'intérêt de séparer synthèse et annexes techniques.
- Rester factuel et non accusatoire : l'audit évalue un système, pas les personnes.
- Respecter la confidentialité du rapport, qui contient potentiellement des informations exploitables par un attaquant (diffusion strictement limitée aux parties autorisées, destruction des données brutes après la période contractuelle convenue).

Ces trois principes méritent d'être développés, car la posture de communication de l'auditeur conditionne fortement la façon dont ses constats seront reçus et suivis d'effet.

Adapter le niveau de détail au public ne signifie pas simplement raccourcir le texte : cela implique de reformuler activement les constats techniques en termes de risque métier compréhensible. Dire à un directeur général qu'une application « présente une vulnérabilité SQLi CVSS 9.8 » a beaucoup moins

d'impact décisionnel que dire qu'« un attaquant externe pourrait, sans authentification, consulter ou modifier l'intégralité de la base de données clients, avec un risque de sanction réglementaire et d'atteinte à la réputation en cas de fuite ».

Rester factuel et non accusatoire est un principe déontologique central de l'audit. Un rapport rédigé sur un ton accusatoire ou moralisateur (« l'équipe n'a manifestement jamais pris la sécurité au sérieux ») produit généralement l'effet inverse de celui recherché : il braque les équipes concernées, nuit à la qualité de la collaboration lors des audits suivants, et détourne l'attention du fond (les constats et leur correction) vers la forme (le ressenti d'être mis en cause personnellement). La formulation recommandée reste toujours centrée sur le système et le processus, jamais sur les personnes : « le processus de gestion des correctifs ne couvre pas l'ensemble du parc serveur » plutôt que « l'administrateur système n'a pas appliqué les correctifs ».

Respecter la confidentialité du rapport est une obligation à la fois contractuelle et éthique évidente, mais dont les modalités pratiques méritent d'être anticipées dès le cadrage de la mission (chapitre 1) : qui a le droit de recevoir le rapport complet, avec ses preuves de concept techniques exploitables, et qui ne devrait recevoir que la synthèse ? Sous quelle forme et pendant combien de temps les données brutes de l'audit (captures, exports de scan, journaux collectés) sont-elles conservées, et selon quelles modalités sont-elles détruites à l'issue de la période contractuelle convenue ? Ces questions, souvent traitées trop tardivement, devraient idéalement être actées dès le mandat initial plutôt qu'au moment de la remise du rapport.

Approfondissement : la restitution orale, un exercice à part entière

Au-delà du document écrit, la restitution orale du rapport (réunion de présentation à la direction et aux équipes techniques) est un moment clé souvent sous-préparé. Une bonne restitution anticipe les questions et objections probables, prépare une hiérarchisation claire des trois à cinq messages essentiels à faire passer (plutôt que de dérouler mécaniquement l'ensemble des constats), et laisse une place réelle au dialogue plutôt qu'à une simple lecture du document. C'est souvent lors de cette restitution que se joue l'adhésion réelle de l'organisation au plan de remédiation proposé.

À retenir

- Un rapport d'audit efficace sépare clairement synthèse décisionnelle et détails techniques, et s'adresse à des publics aux besoins de lecture différents (direction, équipes techniques, auditeurs de certification).
- Chaque constat doit être actionnable et autoportant : criticité contextualisée, preuve, impact métier, recommandation et, idéalement, effort estimé.
- La criticité doit toujours combiner vraisemblance et impact métier dans une matrice de risque, jamais se limiter à un score technique brut trié mécaniquement.
- Le plan de remédiation transforme chaque recommandation en action suivie, avec un responsable nommé, une échéance réaliste et différenciée, et éventuellement des mesures compensatoires ; les risques non corrigés doivent être formellement acceptés plutôt que simplement ignorés.
- Le contre-audit et le suivi transforment l'audit en amélioration continue plutôt qu'en photographie isolée ; il doit vérifier non seulement la correction mais aussi sa pérennité.

- La posture de communication — adaptation au public, ton factuel et non accusatoire, confidentialité rigoureuse — conditionne autant l'impact réel de l'audit que la qualité technique des constats eux-mêmes.

Dr. Zakaria Sawadogo — École Polytechnique de Ouagadougou